

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Docente:
Prof. José Canelón, Ph.D.

UNIDAD III

REDES NEURONALES CONVOLUCIONALES (CNN — CONVOLUTIONAL NEURAL NETWORKS)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Contenido

- Convolución 2D: De MLP a CNN · operación · padding · stride · pooling · feature maps · receptive field
- Arquitecturas históricas: LeNet · AlexNet · ZFNet · VGGNet · GoogLeNet/Inception
- Arquitecturas modernas: ResNet · DenseNet · MobileNet · EfficientNet · ConvNeXt
- Transfer Learning: Feature extraction · fine-tuning · LRs diferenciales · domain adaptation
- Data Augmentation Avanzado: torchvision.transforms v2 · Albumentations · Mixup · CutMix · RandAugment · TTA
- Visualización e interpretabilidad: Filtros · saliency maps · Grad-CAM · CAM · LIME · Integrated Gradients

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Convolución 2D

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

De MLP a CNN — Motivación

El problema con las MLPs para imágenes

- Imagen $224 \times 224 \times 3$: 150,528 píxeles de entrada
- Una capa FC ($150K \rightarrow 1000$) tiene 150M parámetros — inmanejable
- Las MLPs tratan cada píxel de forma independiente: no capturan estructura espacial

De MLP a CNN — Motivación

Lo deseable de una arquitectura para visión

- Que se compartan los pesos: el mismo detector de bordes sirve en cualquier región
- Localidad: cada neurona procesa solo una región pequeña (receptive field)
- Invarianza traslacional: un gato es un gato esté donde esté

La solución: convolución

- Aplicar el mismo filtro deslizando sobre toda la imagen
- Parámetros: solo el filtro (ej. $3 \times 3 = 9$ pesos) reutilizado en cada posición
- CNN: parámetros independientes de la resolución de la imagen

Redes Convolucionales

Son un tipo de red neuronal especializada en el procesamiento de datos que tienen una estructura de rejilla. Por ejemplo:

- Series de tiempo, que pueden considerarse como una rejilla 1D, cuyas muestras se adquieren a intervalos de tiempo regulares
- Imágenes, que son rejillas 2D de píxeles

Su nombre se debe a que aplican la operación de convolución en, al menos, una de sus capas

Las redes convolucionales han sido muy exitosas en aplicaciones prácticas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Capa Convolutiva — Convolución 2D

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

=

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Capa Convolucional — Convolución 2D

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

=

0			

$$10 \times 1 + 10 \times 1 + 10 \times 1 + 10 \times 0 + 10 \times 0 + 10 \times 0 + 10 \times -1 + 10 \times -1 + 10 \times -1 = 0$$

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

=

0	30		

$$10 \times 1 + 10 \times 1 + 10 \times 1 + 10 \times 0 + 10 \times 0 + 10 \times 0 + 0 \times -1 + 0 \times -1 + 0 \times -1 = 30$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Capa Convolutiva — Convolución 2D

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

 $*$

1	0	-1
1	0	-1
1	0	-1

 $=$

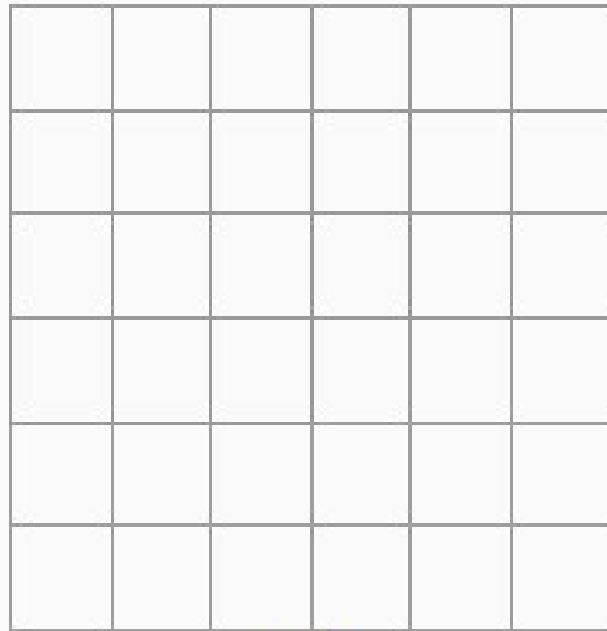
0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

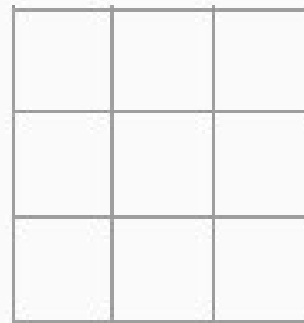
Capa Convolutiva — Convolución 2D

- Dada una entrada de tamaño (h, w) y un filtro de tamaño $F \times F$, el tamaño de la salida es $(h - F + 1, w - F + 1)$



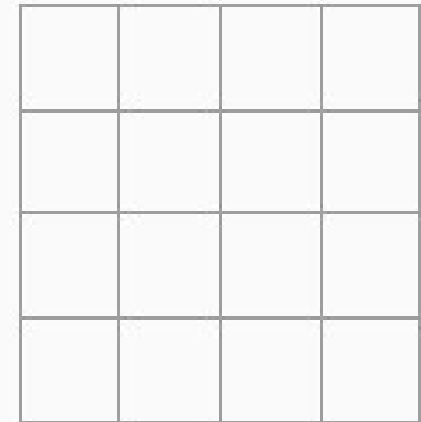
6x6

*



3x3

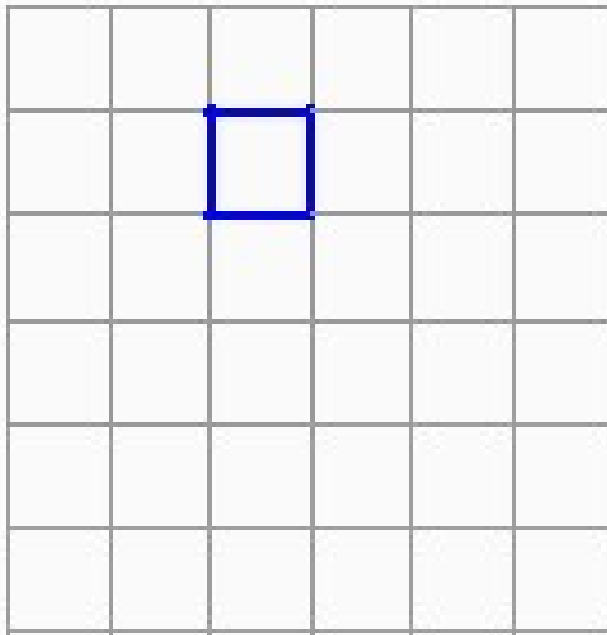
=



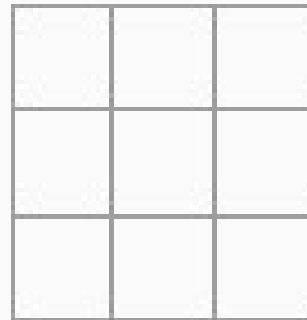
4x4

Capa Convolutiva — Convolución 2D

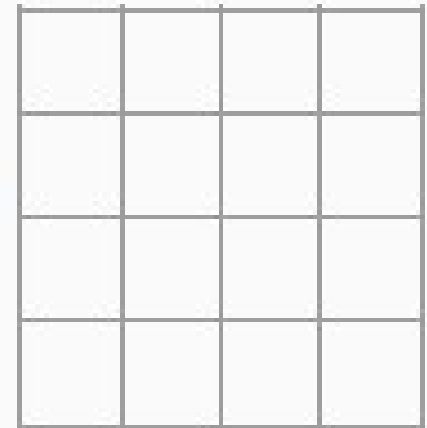
- Los pixeles de las esquinas se usan menos



•

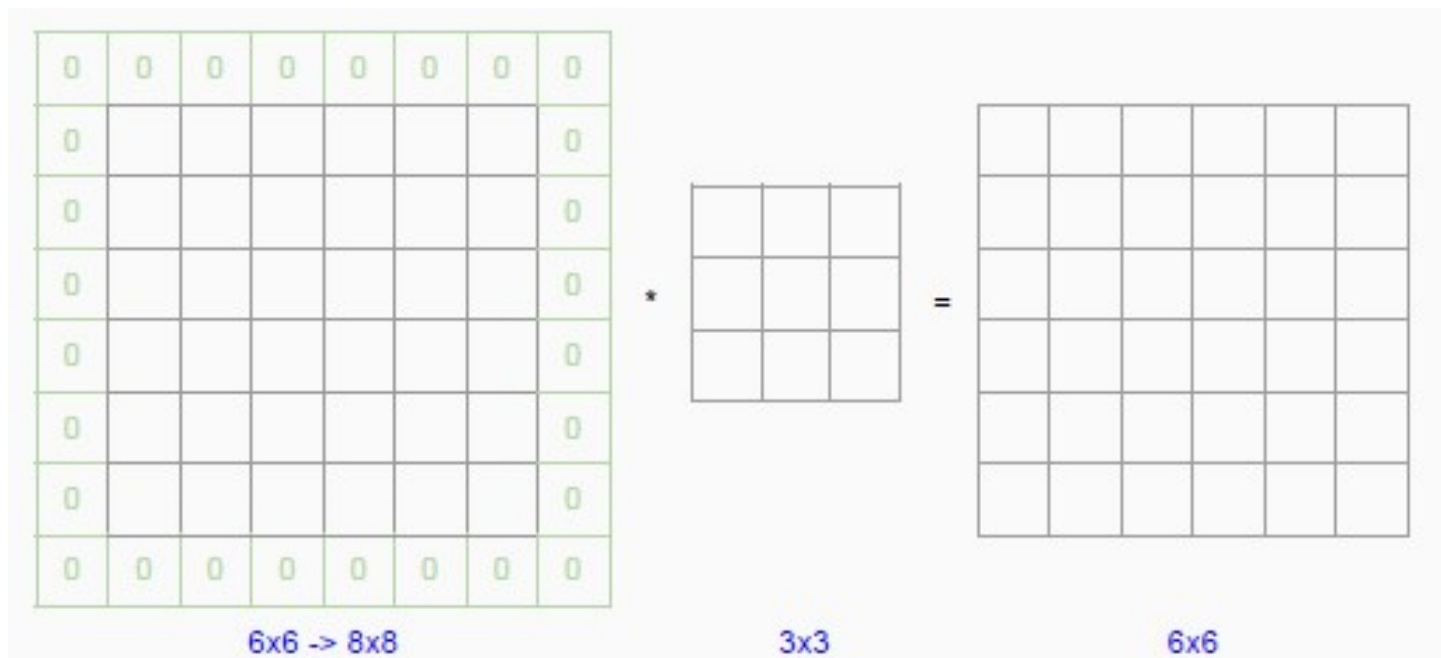


=



Capa Convolutiva — Convolución 2D

- Padding: Relleno
- Dada una entrada de tamaño (h, w) , un filtro de tamaño $F \times F$ y un padding $2p$, el tamaño de la salida es
 $(h + 2p - F + 1, w + 2p - F + 1)$



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Capa Convolutiva — Convolución 2D

¿Por qué padding?

- Sin padding, cada convolución reduce las dimensiones
- Con 10 capas de $Conv(3 \times 3)$ sin padding:
$$224 \times 224 \rightarrow 204 \times 204 \rightarrow 184 \times 184 \rightarrow \dots$$
- Los bordes de la imagen contribuyen menos (procesados por menos filtros)

Padding válido ('valid') — $P=0$

- Útil cuando queremos reducir dimensiones de forma controlada

Padding igual ('same') — $P=(F-1)/2$

- La salida tiene el mismo tamaño que la entrada (con stride $S=1$)
- Mantiene la resolución espacial a través de capas

Tipos de relleno

- Zero padding: añade ceros en los bordes (más común, por defecto en PyTorch)
- Reflect padding: refleja los valores del borde — útil en imágenes naturales

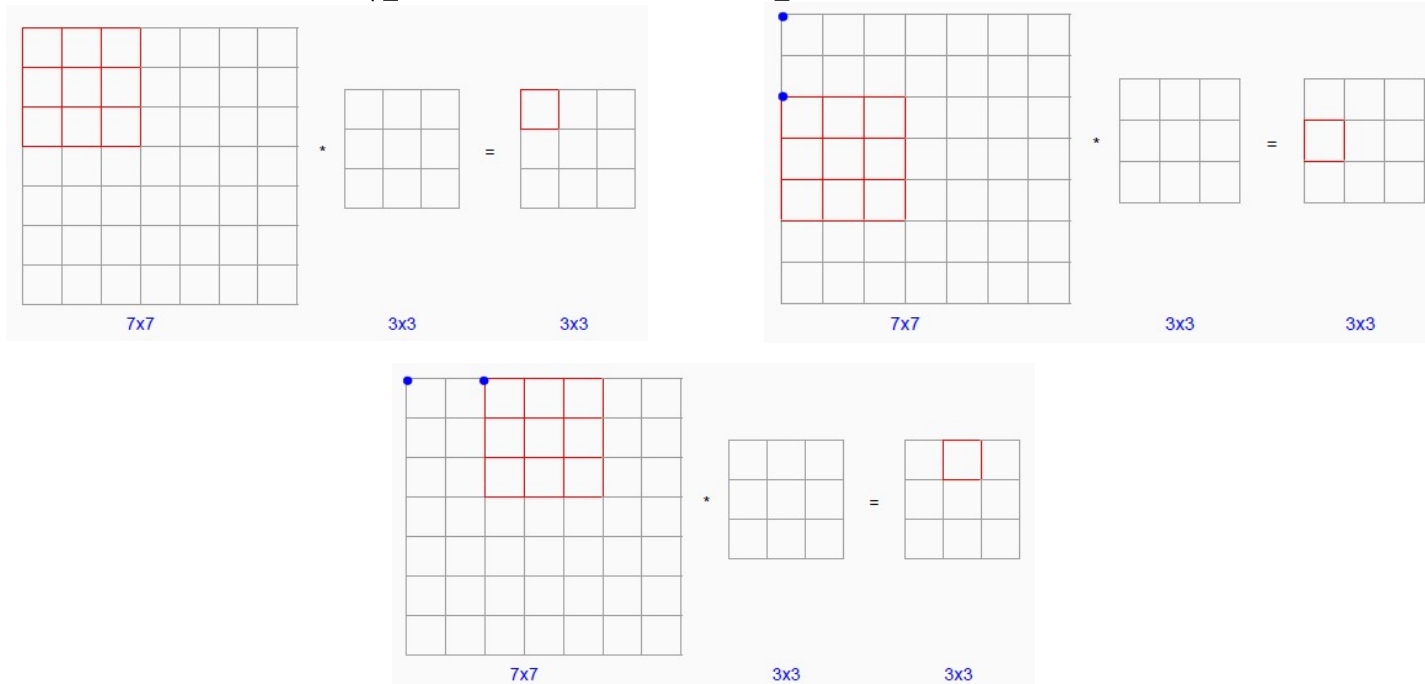
INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Capa Convolutiva — Convolución 2D

- Stride: Paso
- Tamaño de la imagen de salida

$$(h_{out}, w_{out}) = \left(\left\lfloor \frac{h_{in} + 2p - F}{s} \right\rfloor + 1, \left\lfloor \frac{w_{in} + 2p - F}{s} \right\rfloor + 1 \right)$$



Capa Convolutiva — Convolución 2D

Stride

- El stride S es el paso con que el filtro se desplaza sobre la imagen
- $S = 1$: máxima resolución, salida casi igual en tamaño a la entrada
- $S = 2$: reduce h y w a la mitad $\rightarrow$ downsampling sin parámetros extra

Stride asimétrico

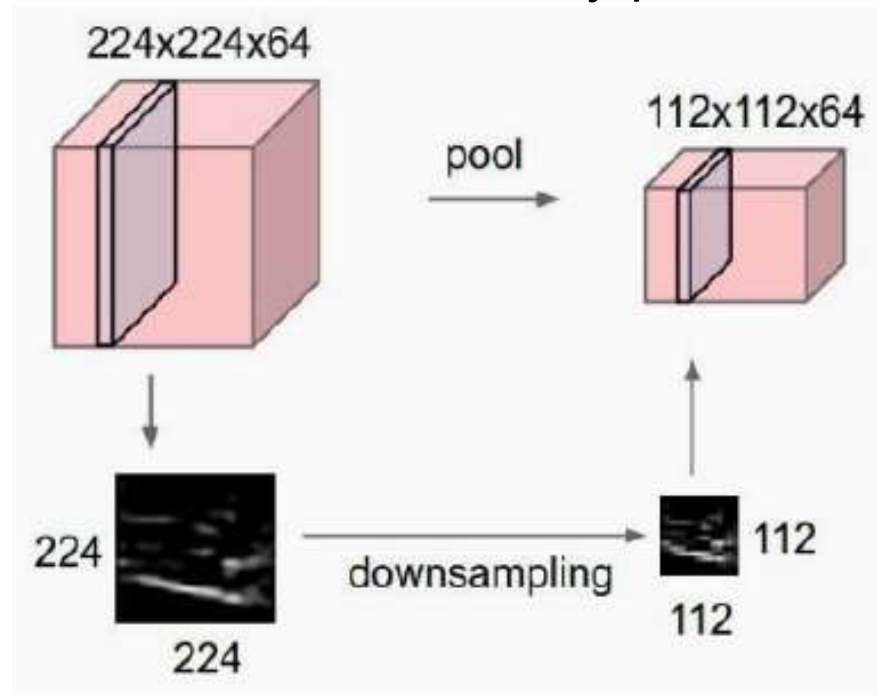
- Se puede usar $S = (2,1)$ para bajar solo en una dimensión (útil en audio/series)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Capa de Pooling

- Hace las representaciones más pequeñas y manejables
- Ayuda a evitar el sobreajuste
- Trabaja de forma independiente sobre cada feature map
- Debe especificarse tamaño del filtro y paso



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

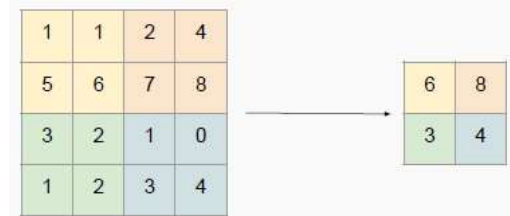
Max Pooling

Definición

- Divide la feature map en regiones no solapadas y toma el máximo de cada una
- Sin parámetros aprendibles: operación fija

MaxPool(2×2, stride=2) — el más común

- Reduce h y w a la mitad
- Invarianza traslacional local: el máximo no cambia con pequeñas traslaciones
- Dimensiones igual que en convolución sin padding



Por qué max y no promedio

- El máximo preserva la activación más fuerte → 'hay un borde aquí'
- El promedio diluye la señal, útil para suavizar pero pierde picos locales

Tendencia actual

- Max pooling está siendo reemplazado por Conv(stride=2) en redes modernas
- ResNet-50+: usa stride en convoluciones en lugar de pooling separado
- ConvNeXt: usa stride=4 en el stem y stride=2 en los downsampling blocks

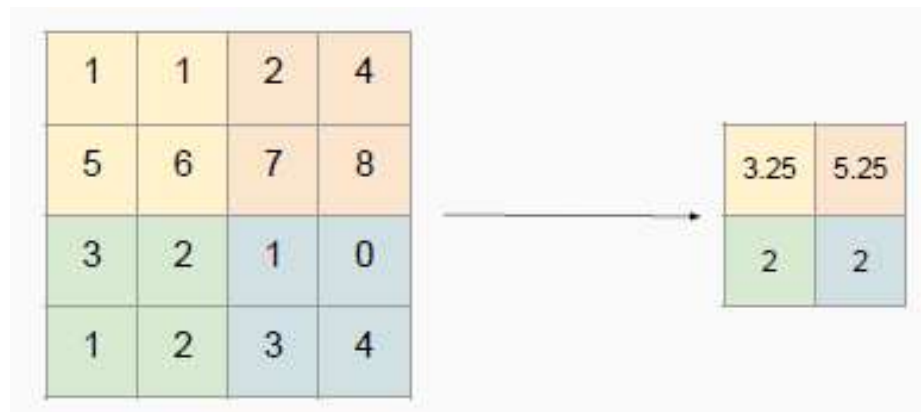
INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Average Pooling

Average Pooling

- Calcula el promedio de cada región
- Más suave que max pooling, menos agresivo con las activaciones
- Usado en LeNet-5 y GoogLeNet original



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Global Average Pooling

- Caso especial: ventana = tamaño completo del feature map
- $(N, C, h, w) \rightarrow (N, C)$ — un escalar por canal
- En PyTorch: `nn.AdaptiveAvgPool2d(1)` o `x.mean(dim=[2,3])`

Ventaja clave de GAP

- Elimina la capa FC final — de miles de parámetros a cero
- Independiente de la resolución de entrada (red completamente convolucional)
- Regularización implícita: no puede hacer overfit espacialmente

GAP en arquitecturas modernas

- ResNet: tras el último bloque residual $\rightarrow$ GAP $\rightarrow$ FC(num_classes)
- EfficientNet, ConvNeXt: mismo patrón — el GAP es estándar desde 2014

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Global Average Pooling

- Sin GAP

Última capa conv: (N, 512, 7, 7)

↓ Flatten

(N, 25,088) ← $512 \times 7 \times 7$

↓ FC(25088 → 4096)

Parámetros: $25,088 \times 4096 = 102,760,448$ ← ~103 millones

↓ FC(4096 → 1000)

Parámetros: $4,096 \times 1000 = 4,096,000$ ← ~4 millones

- Con GAP

Última capa conv: (N, 512, 7, 7)

↓ GAP — promedia cada feature map → un escalar

(N, 512) ← un número por canal, independiente de h y w

↓ FC(512 → 1000)

Parámetros: $512 \times 1000 = 512,000$ ← medio millón

Propiedades de la Convolución

Pesos compartidos (weight sharing)

- El mismo filtro se aplica en todas las posiciones
 - Número de parámetros de una capa convolucional con C_{in} canales de entrada (canales de la imagen) y C_{out} filtros de tamaño $F \times F$

$$Params = C_{out} \times C_{in} \times F \times F + C_{out}$$

- Número de parámetros de la capa FC correspondiente

$$Params = C_{in} \times h_{in} \times w_{in} \times C_{out} \times h_{out} \times w_{out} + C_{out} \times h_{out} \times w_{out}$$

- Ejemplo: Supongamos imágenes de entrada de 224×224
Conv(3→64, 3×3)

$$Params = 64 \times 3 \times 3 \times 3 + 64 = 1792 \text{ (Independiente de } h \text{ y } w)$$

Capa FC equivalente

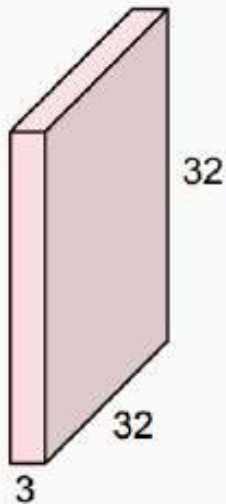
$$Params = 3 \times 224 \times 224 \times 64 \times 222 \times 222 + 64 \times 222 \times 222 \approx 2.1 \times 10^{12}$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

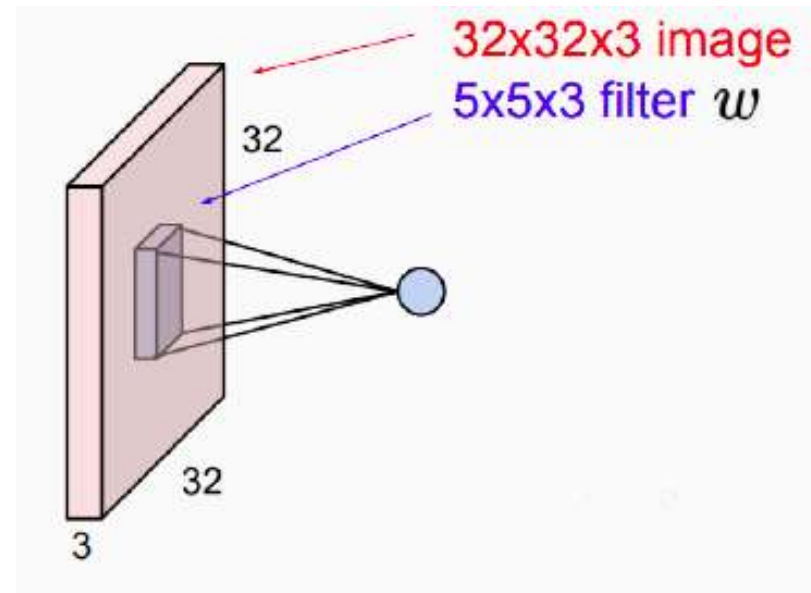
Unidad III

Capa Convolutiva — Convolución 2D

32x32x3 image



5x5x3 filter



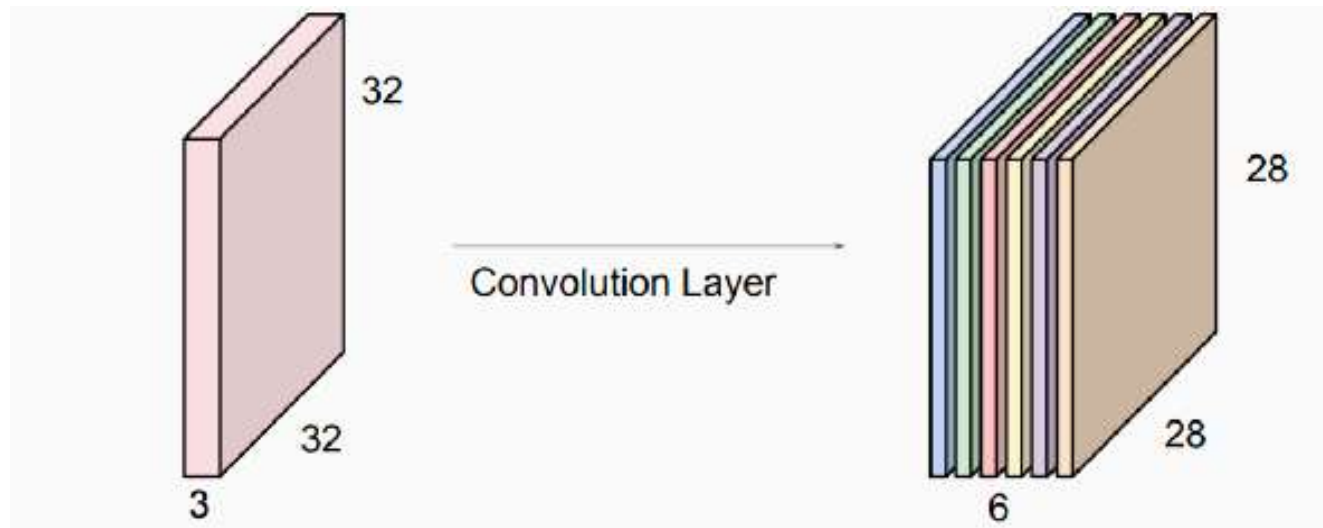
Total de parámetros del filtro = 76 = 75 ($5 \times 5 \times 3$) + 1 (bias)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Capa Convolutiva — Convolución 2D

- Por ejemplo, si tenemos 6 filtros de tamaño 5×5 , tenemos 6 feature maps



- Se obtiene una nueva “imagen” de $28 \times 28 \times 6$

Propiedades de la Convolución

Localidad (local connectivity)

- Cada neurona en la feature map está conectada solo a una región $F \times F$ de la entrada
 - Cada valor en el feature map de salida se calcula aplicando el filtro a una pequeña región de la entrada — no a toda la imagen
- Esto contrasta con una capa FC, donde cada neurona de salida está conectada a todos los píxeles de la entrada. En la CNN, cada neurona ignora todo lo que está fuera de su ventana $F \times F$.
- Consecuencias prácticas:
 - Muchos menos parámetros por neurona
 - El modelo aprende detectores de patrones locales (bordes, esquinas, texturas) en vez de patrones que dependen de píxeles arbitrariamente distantes entre sí — En imágenes naturales la información relevante es casi siempre local

Propiedades de la Convolución

Equivarianza traslacional

- Si la entrada se traslada, la feature map se traslada en la misma cantidad $T[f(x)] = f[T(x)]$ donde T es una traslación

Composición jerárquica

- Capas tempranas detectan patrones simples (bordes, gradientes)
- Capas profundas combinan patrones simples → detectores de objetos complejos
- Esta jerarquía emerge automáticamente del entrenamiento

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Propiedades de la Convolución Composición Jerárquica

Preview

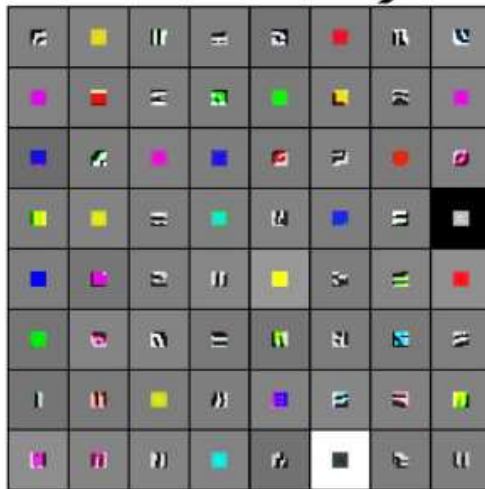


Low-level
features

Mid-level
features

High-level
features

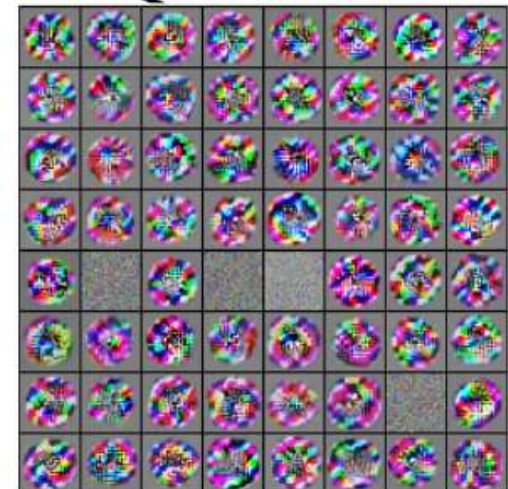
Linearly
separable
classifier



VGG-16 Conv1_1



VGG-16 Conv3_2



VGG-16 Conv5_3

[Zeiler and Fergus 2013]

Visualization of VGG-16 by Lane McIntosh. VGG-16 architecture from [Simonyan and Zisserman 2014].

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Entrada Multicanal y Feature Maps

Tensor de entrada

- Imagen RGB: $(N, 3, h, w)$ — N =batch, 3=canales RGB, $h \times w$ =resolución

Filtros 3D para entrada multicanal

- Un filtro tiene forma (C_{in}, F, F) — opera en todos los canales a la vez
- Con C_{out} filtros: tensor de pesos $\in \mathbb{R}^{C_{out} \times C_{in} \times F \times F}$
- Salida: $(N, C_{out}, h_{out}, w_{out})$

Feature map

- Cada uno de los C_{out} canales de salida es un 'feature map'
- Representa la respuesta de un filtro en cada posición espacial
- Alta activación = el patrón del filtro está presente en esa región

Visualización

- Primera capa: feature maps $\approx$ imágenes en escala de grises filtradas
- Capas profundas: feature maps son abstractos, no directamente interpretables

Receptive Field — Definición

Definición

- El receptive field (RF) de una neurona es la región de la imagen original que influye en su activación
- Con stride $S = 1$, cada capa suma $F - 1$ al RF anterior
- Con stride $S > 1$, el RF de capas posteriores se amplía
 - Con $S = 2$, cada posición de la siguiente capa ve 2X más de la entrada

RF efectivo vs RF teórico

- En la práctica solo el centro del RF tiene influencia fuerte (distribución gaussiana)
- Las neuronas en el borde del RF tienen gradientes casi nulos

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Receptive Field —Cálculo

¿Por qué cada capa suma $F-1$ al Receptive Field?

Al tomar F neuronas consecutivas de la capa anterior, sus ventanas se solapan — solo $F - 1$ posiciones nuevas de la imagen original quedan al descubierto

Ejemplo

Capa 1 — filtro $F=3$ sobre la imagen

Imagen: [1] [2] [3] [4] [5] [6] [7]
 ↑↑↑
 filtro de 3 → RF = 3

- El solapamiento entre ventanas consume $F - 1$ posiciones en el centro
- Solo $F - 1$ posiciones nuevas quedan expuestas en el extremo

Capa 2 — filtro $F=3$ sobre el output de Capa 1

o1 cubre píxeles: [1 2 3]
o2 cubre píxeles: [2 3 4]
o3 cubre píxeles: [3 4 5]

Solo $F - 1 = 2$ píxeles nuevos (4 y 5)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Receptive Field —Cálculo

Fórmula general

$$RF = 1 + L \times (F - 1)$$

- Cada capa suma $F-1$ píxeles al RF acumulado
- El 1 es el punto de partida: una neurona en la imagen ve exactamente 1 píxel

Acumulación capa a capa (Conv 3×3 , $S=1$)

Capa 0 (imagen): $RF = 1$

Capa 1: $RF = 1 + 2 = 3$

Capa 2: $RF = 3 + 2 = 5$

Capa 3: $RF = 5 + 2 = 7$

- Cada capa suma exactamente $F-1 = 2$ al RF anterior

$2 \times \text{Conv}(3 \times 3) = 1 \times \text{Conv}(5 \times 5)$

1 capa Conv(5×5): $RF = 1 + 1 \times 4 = 5$

2 capas Conv(3×3): $RF = 1 + 2 \times 2 = 5$ ✓

- Mismo RF, menos parámetros y una no-linealidad extra

$1 \times \text{Conv}(5 \times 5): 25C^2$

$2 \times \text{Conv}(3 \times 3): 18C^2$ (-28%)

5 capas Conv(3×3)

$$RF = 1 + 5 \times 2 = 11$$

- Una neurona ve 11×11 px sin filtros mayores de 3×3
- La profundidad sustituye al tamaño del filtro para ampliar el contexto

¿Por Qué Filtros Pequeños (3×3)?

Equivalencia de receptive field

- 2 capas Conv(3×3) tienen RF=5 — igual que 1 capa Conv(5×5)
- 3 capas Conv(3×3) tienen RF=7 — igual que 1 capa Conv(7×7)

Ventaja 1: menos parámetros

- $2 \times \text{Conv}(3 \times 3, C \rightarrow C): 2 \times C^2 \times 9 = 18C^2$
- $1 \times \text{Conv}(5 \times 5, C \rightarrow C): C^2 \times 25 = 25C^2$
- Reducción del 28% de parámetros para mismo RF

Ventaja 2: más no-linealidades

- 2 capas = 2 activaciones ReLU → mayor capacidad expresiva
- Con 1 capa Conv(5×5): solo 1 activación no lineal

Por Qué Filtros Pequeños (3×3)

Principio popularizado por VGG (Simonyan & Zisserman, 2014)

- VGG usa exclusivamente filtros 3×3 en toda la red
- Todos los modelos modernos siguieron este principio

Excepción: el stem (Bloque inicial de la red, procesa la imagen cruda)

- Primera capa con entrada RGB: Conv(7×7, S=2) en ResNet — el RF grande inicial reduce la resolución rápidamente, luego solo Conv(3×3)

Convoluciones Dilatadas (Atrous)

Motivación

- Produce RF grande SIN reducir resolución espacial y SIN más parámetros
- Crítico en segmentación semántica: se necesita contexto global + resolución alta

Mecanismo

- Se insertan $d - 1$ ceros entre elementos adyacentes del filtro
- Con $F = 3$ y dilation d : RF efectivo = $2d + 1$
- $d=1$ (estándar): RF=3 · $d=2$: RF=5 · $d=4$: RF=9 · $d=8$: RF=17

Aplicaciones

- DeepLab (v1-v3): ASPP usa múltiples tasas de dilatación en paralelo
- Segmentación semántica: preserva resolución full mientras amplía contexto

Convoluciones Depthwise Separables

Motivación

- Una $\text{Conv}(C_{in} \rightarrow C_{out}, F \times F)$ opera en espacio y canales simultáneamente
- Separar estas dos operaciones reduce drásticamente el número de parámetros

Depthwise convolution

- Un filtro $F \times F$ por canal de entrada $\rightarrow C_{in}$ filtros independientes
- Parámetros: $C_{in} \times F^2$ (en lugar de $C_{in} \times C_{out} \times F^2$)

Pointwise convolution

- $\text{Conv}(1 \times 1)$ para mezclar los canales: $C_{in} \rightarrow C_{out}$
- Parámetros: $C_{in} \times C_{out}$

Convoluciones Depthwise Separables

Comparativa de parámetros ($F = 3$)

- Conv estándar: $C_{in} \times C_{out} \times 9$
- Depthwise separable: $C_{in} \times 9 + C_{in} \times C_{out} = C_{in}(9 + C_{out})$
- Reducción $\approx 8 - 9X$ para C_{out} típico (64 – 256)

Uso en arquitecturas

- MobileNet (Howard et al., 2017): usa depthwise separables en toda la red
- Xception (Chollet, 2017): reformula Inception en depthwise separables
- ConvNeXt: usa depthwise 7×7 para el RF sin el costo de Conv(7×7) estándar

Convolución 1×1 — El 'Network in Network'

¿Qué hace una Conv(1×1)?

- No captura contexto espacial ($F=1$): solo mezcla canales
- Equivalente a aplicar una capa FC a cada posición espacial independientemente

Usos fundamentales

- Cambiar el número de canales: $C_{in} \rightarrow C_{out}$ (sin costo espacial)
- Reducción de canales (bottleneck): $C_{in} \rightarrow C_{in}/4 \rightarrow C_{in}/4 \rightarrow C_{in}$
- Agregar no-linealidades entre capas sin aumentar RF

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Convolución 1×1

- Ejemplo: Supongamos un tensor de 256 canales al que se desea aplicar una Conv(3×3) para obtener 256 canales de salida:

- Sin bottleneck

$$\text{Conv}(256 \rightarrow 256, 3 \times 3): 256 \times 256 \times 9 = 589,824 \text{ parámetros}$$

- Con bottleneck: En lugar de aplicar la Conv(3×3) directamente sobre los 256 canales, primero se comprimen a 64 con una Conv(1×1), luego se aplica la Conv(3×3) en ese espacio reducido, y finalmente se restauran los 256 canales con otra Conv(1×1)

$$\text{Conv}(256 \rightarrow 64, 1 \times 1): 256 \times 64 \times 1 = 16,384$$

$$\text{Conv}(64 \rightarrow 64, 3 \times 3): 64 \times 64 \times 9 = 36,864$$

$$\text{Conv}(64 \rightarrow 256, 1 \times 1): 64 \times 256 \times 1 = 16,384$$

$$\text{Total:} \quad \quad \quad = 69,632 \text{ parámetros}$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Convolución 1×1

En arquitecturas modernas

- ResNet bottleneck: Conv(1×1) baja canales, Conv(3×3) extrae features, Conv(1×1) sube
- MobileNetV2: punto de entrada y salida de cada bloque invertido
- GoogLeNet: reduce canales antes de cada conv costosa en el módulo Inception

Parámetros: $C_{in} \times C_{out}$ — muy eficiente

- Conv(256→64, 1×1): 16,384 parámetros
- Comparado con Conv(256→256, 3×3): 589,824 parámetros — 36X más costoso

Batch Normalization en CNNs

Posición en la red

- Estándar en CNNs: Conv $\rightarrow$ BN $\rightarrow$ ReLU
- Alternativa moderna (Pre-Norm): BN $\rightarrow$ Conv $\rightarrow$ ReLU (ResNetV2)

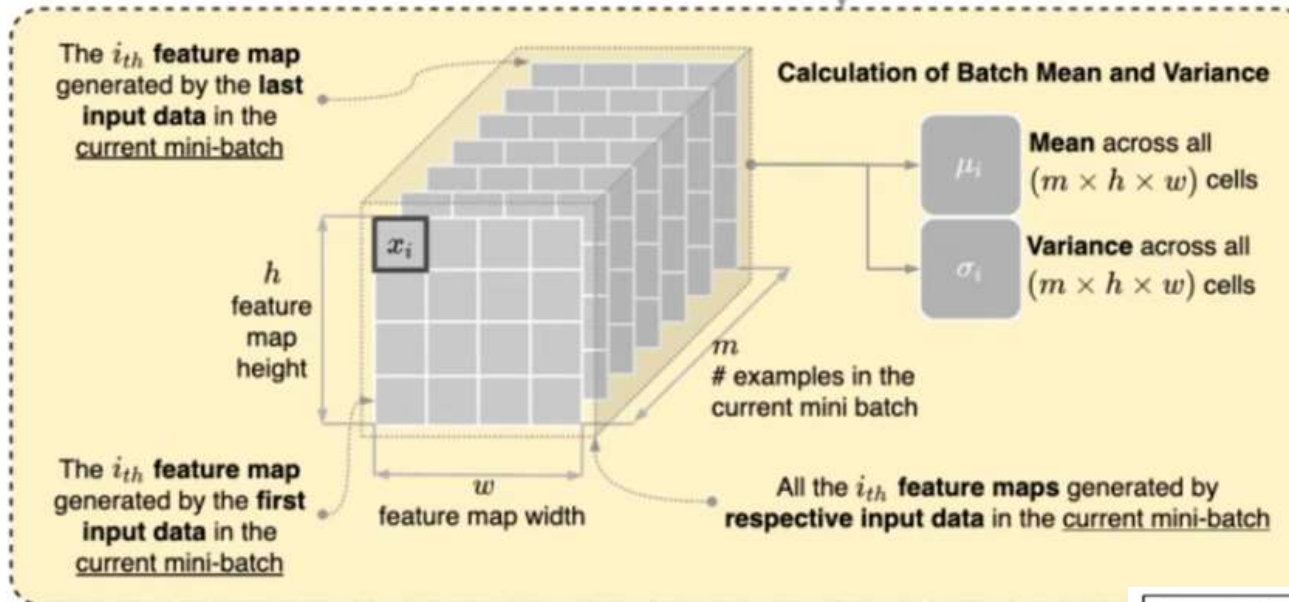
BN en CNN: normalización sobre (N, h, w) por canal

- Para cada canal c : normalizar sobre el batch Y las posiciones espaciales
- Parámetros γ , β de escala y desplazamiento: uno por canal

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Batch Normalization en CNNs



Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Entrenamiento

Proceso general

- Las CNNs se entrenan de extremo a extremo (end-to-end), de forma supervisada
- Se usa un conjunto de datos etiquetados: $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$

Función de costo y optimización

- Promedio de la pérdida individual sobre las m muestras del batch:

$$J = \frac{1}{n} \sum_{i=1}^n L(y^{(i)}, \hat{y}^{(i)})$$

- Función de pérdida (i): entropía cruzada para clasificación, MSE para regresión
- Optimización con descenso de gradiente (SGD, Adam, AdamW) — igual que en MLP
 - El backprop fluye a través de Conv, BN y Pool sin cambios — misma regla de la cadena

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Imagenet

- Base de datos para entrenar modelos de visión por computadora
- 1000 clases
- La data se divide en conjuntos de
 - Entrenamiento: 1,2 millones de imágenes, 1000 (aprox) por clase
 - Validación: 50 mil imágenes, 50 por clase
 - Prueba: 100 mil imágenes

Evolución de las CNNs — Timeline

1989–1998: Las primeras CNNs

- LeCun et al. (1989): primeras CNNs con backpropagation aplicadas a dígitos
- LeNet-5 (1998): primera CNN exitosa en producción — reconocimiento de cheques bancarios

2012: El cambio de paradigma

- AlexNet (Krizhevsky, Sutskever, Hinton): top-5 error 15.3% vs 26.2% (2° lugar)
- GPU (2× GTX 580) + ReLU + Dropout + Data Augmentation — el template moderno

Evolución de las CNNs — Timeline

2013–2014: Profundidad

- ZFNet (2013): visualización de features, error 11.7%
- VGGNet (2014): filtros 3×3 uniformes, 16-19 capas, error 7.3%
- GoogLeNet/Inception (2014): módulos paralelos, error 6.7%

2015–2016: La era residual

- ResNet (He et al., 2016): skip connections, 152 capas, error 3.57% — sub-humano (5%)

2017–2024: Escalado y eficiencia

- MobileNet, EfficientNet, ConvNeXt — precisión vs. eficiencia computacional

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Arquitecturas Históricas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

LeNet-5 (LeCun et al., 1998)

Contexto

- Diseñada para reconocimiento de dígitos escritos a mano en cheques bancarios
- Desplegada en producción por AT&T/Lucent para lectura de dígitos USPS

Parámetros: ~60,000

Innovaciones

- Demostró que CNNs aprenden features jerárquicas automáticamente
- Patrón [Conv $\rightarrow$ Pool] $\times$ N $\rightarrow$ FC que define las CNNs clásicas
- Backpropagation end-to-end sobre toda la red

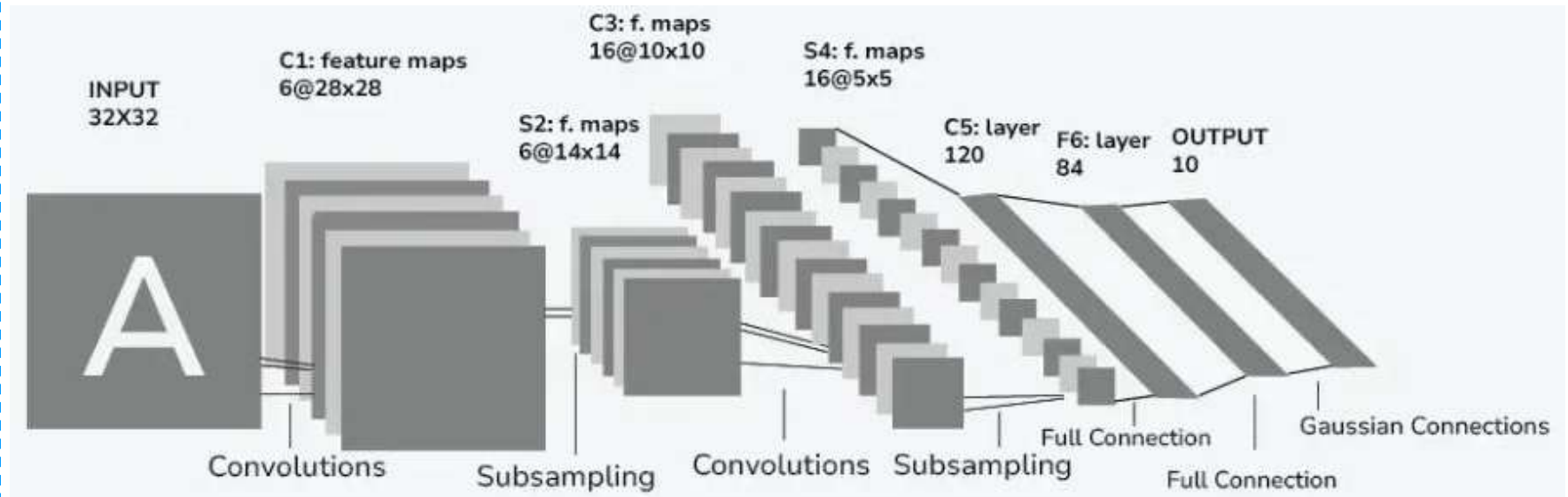
Limitaciones

- tanh/Sigmoid: vanishing gradient en redes más profundas
- No escalable a ImageNet: 60K params insuficientes para 1000 clases

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

LeNet-5 (LeCun et al., 1998)



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

AlexNet — El Momento Histórico (2012)

ILSVRC 2012: el punto de inflexión del DL

- ImageNet: 1.2M imágenes de entrenamiento, 1000 clases, imágenes 224×224
- Antes de AlexNet: los mejores sistemas usaban SIFT + SVM o features manuales

Resultado impactante

- AlexNet: top-5 error 15.3%
- Segundo lugar (métodos clásicos): 26.2% — diferencia de +10.8%
- La brecha era tan grande que el campo tardó en creer los resultados

Los ingredientes del éxito

- GPU: 2× NVIDIA GTX 580 (3GB) — entrenado en paralelo por 5-6 días
- ReLU: convergencia 6× más rápida que tanh (gráfico en el paper)
- Dropout ($p=0.5$): regularización en las capas FC
- Data augmentation: crops, flips horizontales, PCA color jitter

Impacto en el campo

- Inició la era del Deep Learning: financiación, investigación, industria
- Mostró que GPU + grandes datos + arquitectura correcta = breakthrough

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

AlexNet — Arquitectura

Entrada: $224 \times 224 \times 3$

Capas convolucionales

- Conv($3 \rightarrow 96$, 11×11 , $S=4$, $P=0$): $55 \times 55 \times 96 \rightarrow$ MaxPool($3 \times 3, S=2$): $27 \times 27 \times 96$
- Conv($96 \rightarrow 256$, 5×5 , $P=2$): $27 \times 27 \times 256 \rightarrow$ MaxPool($3 \times 3, S=2$): $13 \times 13 \times 256$
- Conv($256 \rightarrow 384$, 3×3 , $P=1$): $13 \times 13 \times 384$
- Conv($384 \rightarrow 384$, 3×3 , $P=1$): $13 \times 13 \times 384$
- Conv($384 \rightarrow 256$, 3×3 , $P=1$): $13 \times 13 \times 256 \rightarrow$ MaxPool($3 \times 3, S=2$): $6 \times 6 \times 256$

Capas densas

- Flatten: $9216 \rightarrow$ FC(4096 , ReLU, Dropout) $\rightarrow$ FC(4096 , ReLU, Dropout) $\rightarrow$ FC(1000)

Total: ~60 millones de parámetros

- FC1: $9216 \times 4096 = 37.7M$ · FC2: $4096 \times 4096 = 16.8M$
- El 90% de los parámetros en las dos primeras capas FC

Nota histórica

- Split en 2 GPUs: la red se dividía por canales entre las dos GPUs
- La comunicación entre GPUs solo ocurría en capas específicas

AlexNet — ReLU y Activaciones

Problema con sigmoid/tanh

- $\sigma(x)$ se satura para $|x| > 3$: gradiente $\approx 0 \rightarrow$ vanishing gradient
- Entrenamiento muy lento en redes profundas

ReLU: $f(x) = \max(0, x)$

- No satura para $x > 0$: gradiente = 1 siempre que $x > 0$
- Computacionalmente barata: solo una comparación
- Sparsidad: $\sim 50\%$ de neuronas en 0 por batch $\rightarrow$ activaciones sparse

Resultado empírico en AlexNet

- ReLU converge 6× más rápido que tanh en CIFAR-10

AlexNet — Dropout y Regularización

En AlexNet

- Aplicado a las dos primeras capas FC ($p=0.5$)
- Sin dropout: el modelo sobreajusta con 60M parámetros

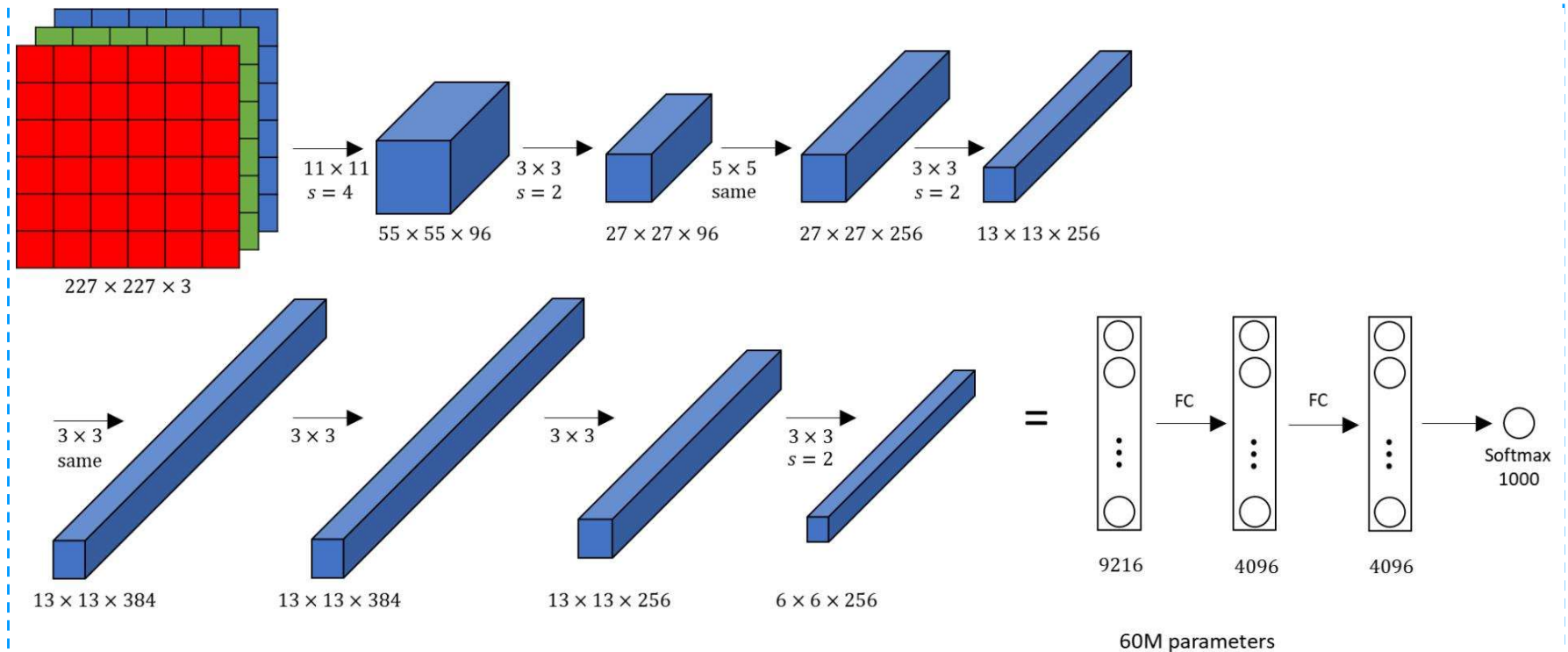
Configuración típica en CNNs modernas

- Dropout solo en capas FC (no en capas conv)
- DropPath / Stochastic Depth: variante para transformers y ConvNeXt
- $p=0.1-0.2$ en capas conv avanzadas; $p=0.3-0.5$ en FC

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

AlexNet



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III



VGGNet — El Principio del Filtro 3×3

Simonyan & Zisserman (2014) — VGG Group, Oxford

- Pregunta: ¿cuánta profundidad se necesita para buen rendimiento?
- Respuesta sistemática: sustituir todos los filtros por 3×3, añadir capas

Principio de diseño

- Todas las convoluciones: 3×3 con $p=1$ (same), $S=1$
- Downsampling: $\text{MaxPool}(2\times 2, S=2)$ — duplica los canales en cada bloque
- Estructura: $[\text{Conv}\times 2-3 \rightarrow \text{MaxPool}] \times 5 \rightarrow \text{FC}\times 3$

Por qué 3×3 es suficiente

- 2 capas $\text{Conv}(3\times 3)$: $\text{RF}=5$, menos parámetros y más no-linealidades que $1\times \text{Conv}(5\times 5)$
- 3 capas $\text{Conv}(3\times 3)$: $\text{RF}=7$, $27C^2$ params vs $49C^2$ con $1\times \text{Conv}(7\times 7)$

Top-5 error en ILSVRC 2014: 7.3% (2° lugar)

- Ganó GoogLeNet con 6.7% — pero VGG fue más influyente en transfer learning

Legado

- Sus pesos pre-entrenados (VGG-16, VGG-19) siguen siendo muy usados
- Arquitectura simple y comprensible — fácil de adaptar

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

VGGNet — Arquitecturas VGG-16 y VGG-19

VGG-16 (entrada $224 \times 224 \times 3$)

- Block1: Conv $\times 2$ ($3 \rightarrow 64$) $\rightarrow$ MaxPool: $112 \times 112 \times 64$
- Block2: Conv $\times 2$ ($64 \rightarrow 128$) $\rightarrow$ MaxPool: $56 \times 56 \times 128$
- Block3: Conv $\times 3$ ($128 \rightarrow 256$) $\rightarrow$ MaxPool: $28 \times 28 \times 256$
- Block4: Conv $\times 3$ ($256 \rightarrow 512$) $\rightarrow$ MaxPool: $14 \times 14 \times 512$
- Block5: Conv $\times 3$ ($512 \rightarrow 512$) $\rightarrow$ MaxPool: $7 \times 7 \times 512$
- Flatten: 25,088 $\rightarrow$ FC(4096) $\rightarrow$ FC(4096) $\rightarrow$ FC(1000)

VGG-19

- Igual que VGG-16 pero blocks 3-5 tienen Conv $\times 4$ en vez de Conv $\times 3$
- Ganancia marginal: 7.3% vs 7.4% — confirma que la profundidad ayuda menos allá de 16

Parámetros

- VGG-16: 138M · VGG-19: 144M
- Capas conv: 15M · Capas FC: 123M — el 89% en las FCs

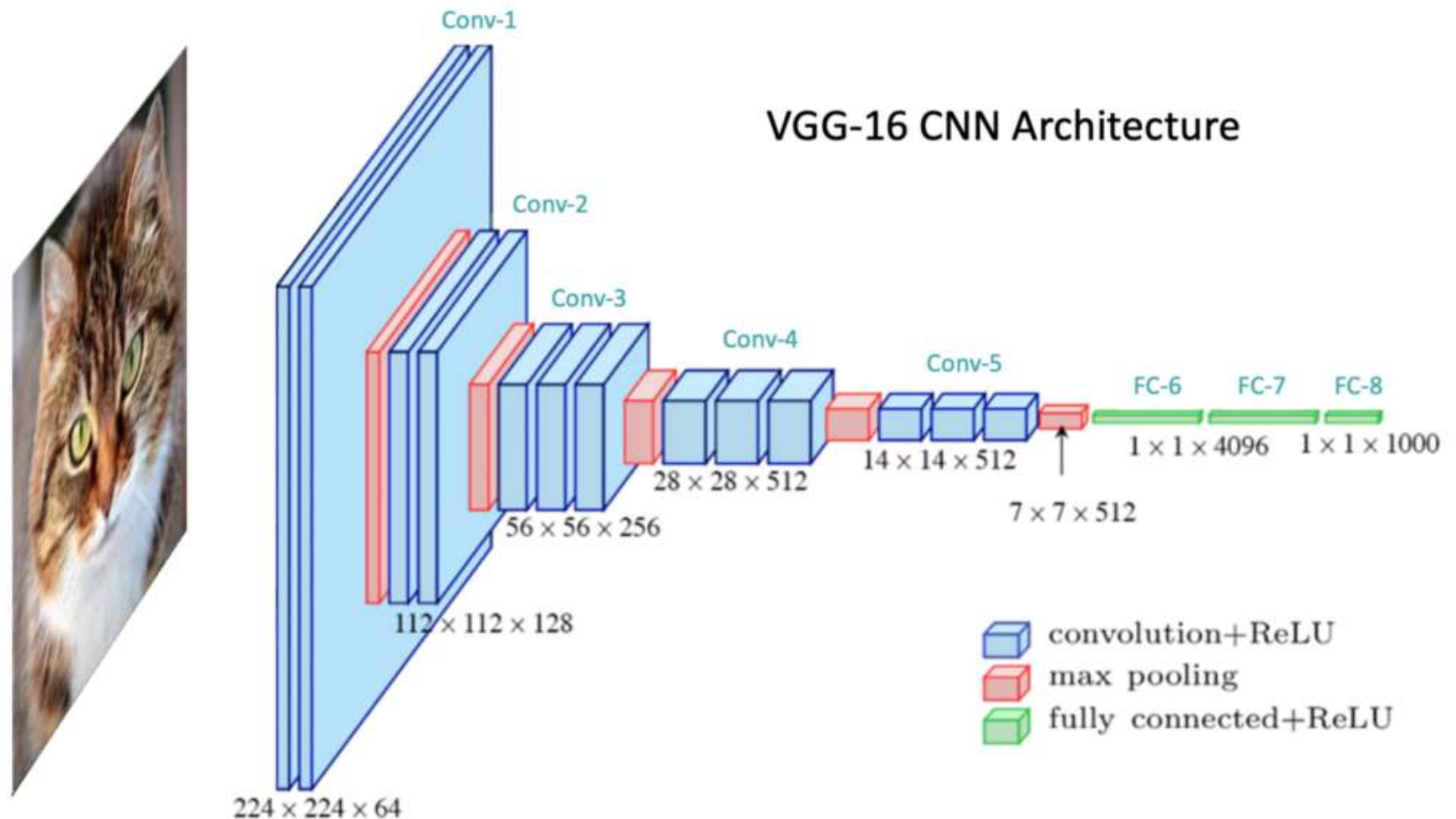
Limitación principal

- 138M params hacen el fine-tuning costoso en GPU con poca VRAM
- Sin skip connections: aún vulnerable a vanishing gradient > 19 capas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Arquitectura VGG-16

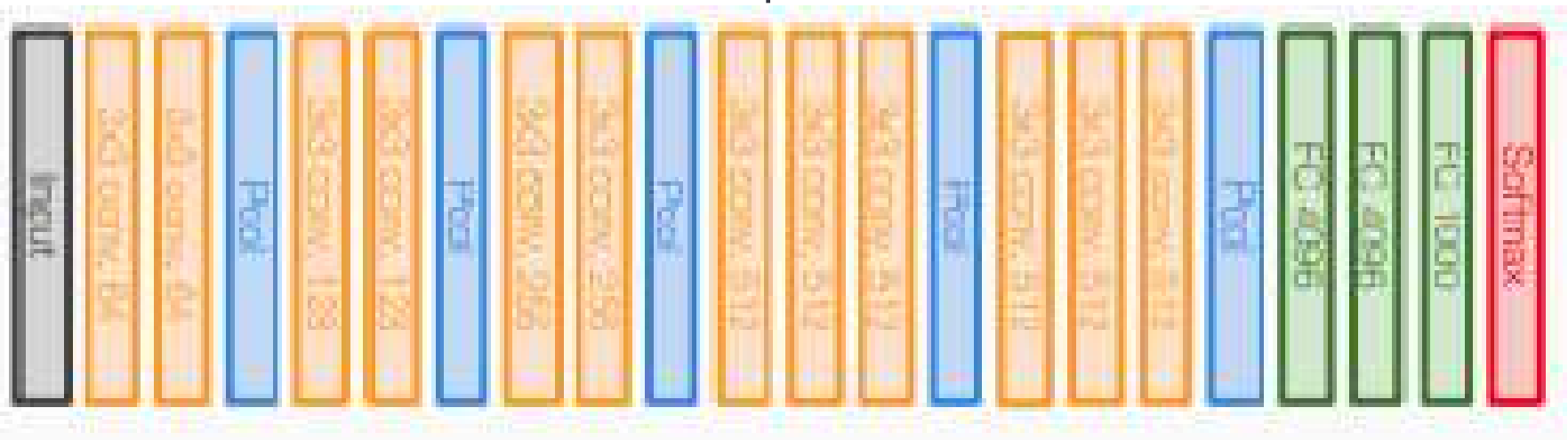


INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Arquitectura VGG-16

138M de parámetros



GoogLeNet / Inception v1 (Szegedy et al., 2014)

Motivación

- ¿Qué tamaño de filtro es óptimo? En vez de elegir, usar todos en paralelo

Módulo Inception

- Rama 1×1 : $Conv(1 \times 1)$
- Rama 3×3 : $Conv(1 \times 1) \rightarrow Conv(3 \times 3)$
- Rama 5×5 : $Conv(1 \times 1) \rightarrow Conv(5 \times 5)$
- Rama MaxPool: $MaxPool(3 \times 3, S = 1, P = 1) \rightarrow Conv(1 \times 1)$
- Concatenar todos los outputs en la dimensión de canales

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

GoogLeNet / Inception v1 (Szegedy et al., 2014)

Los *Conv*(1×1) antes de los filtros grandes

- Reducen canales ANTES de la conv costosa (bottleneck de Inception)
- $\text{Conv}(256 \rightarrow 32, 1 \times 1) \rightarrow \text{Conv}(32 \rightarrow 32, 5 \times 5)$: $256 \times 32 + 32 \times 32 \times 25 = 33\text{K}$ vs $256 \times 32 \times 25 = 205\text{K}$

Arquitectura GoogLeNet

- 22 capas con peso · 9 módulos Inception en cascada
- Sin capas FC al final (usa GAP) → solo 5M parámetros totales vs 60M de AlexNet

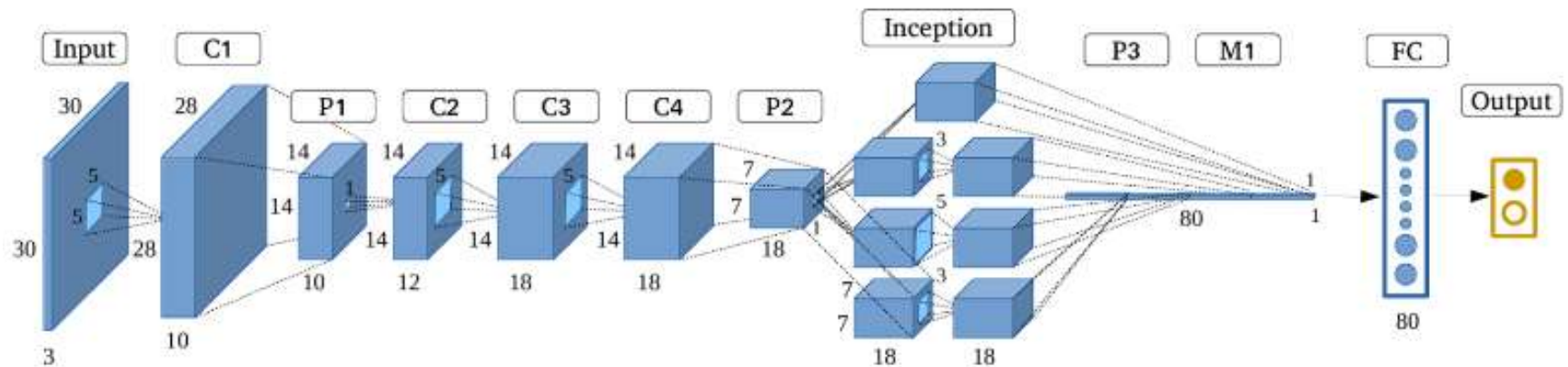
ILSVRC 2014: top-5 error 6.67% — primer lugar

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

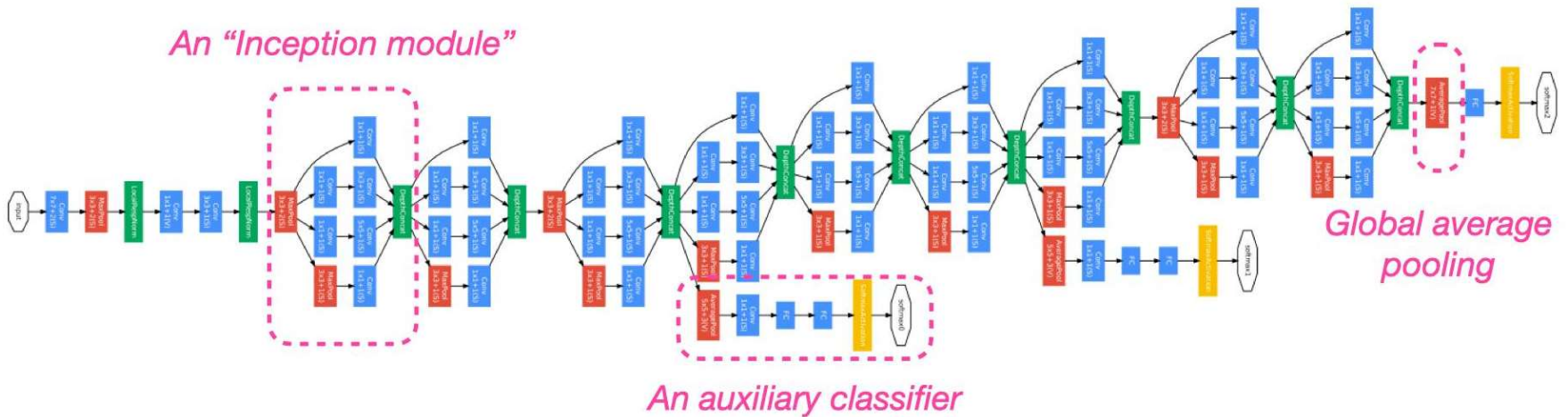
Unidad III

GoogleNet

6.8M de parámetros



An "Inception module"



Global average pooling

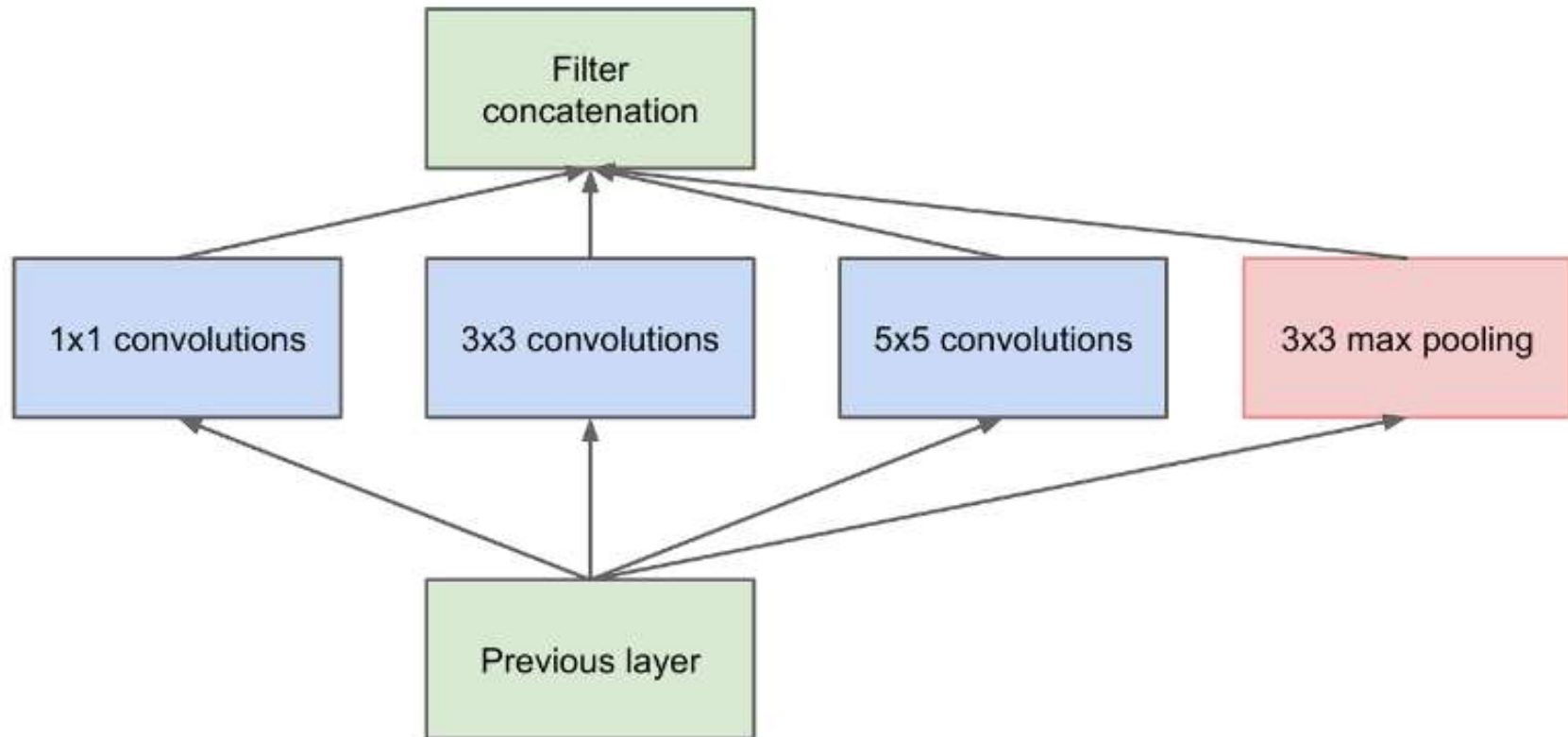
An auxiliary classifier

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

GoogleNet

Módulo Inception

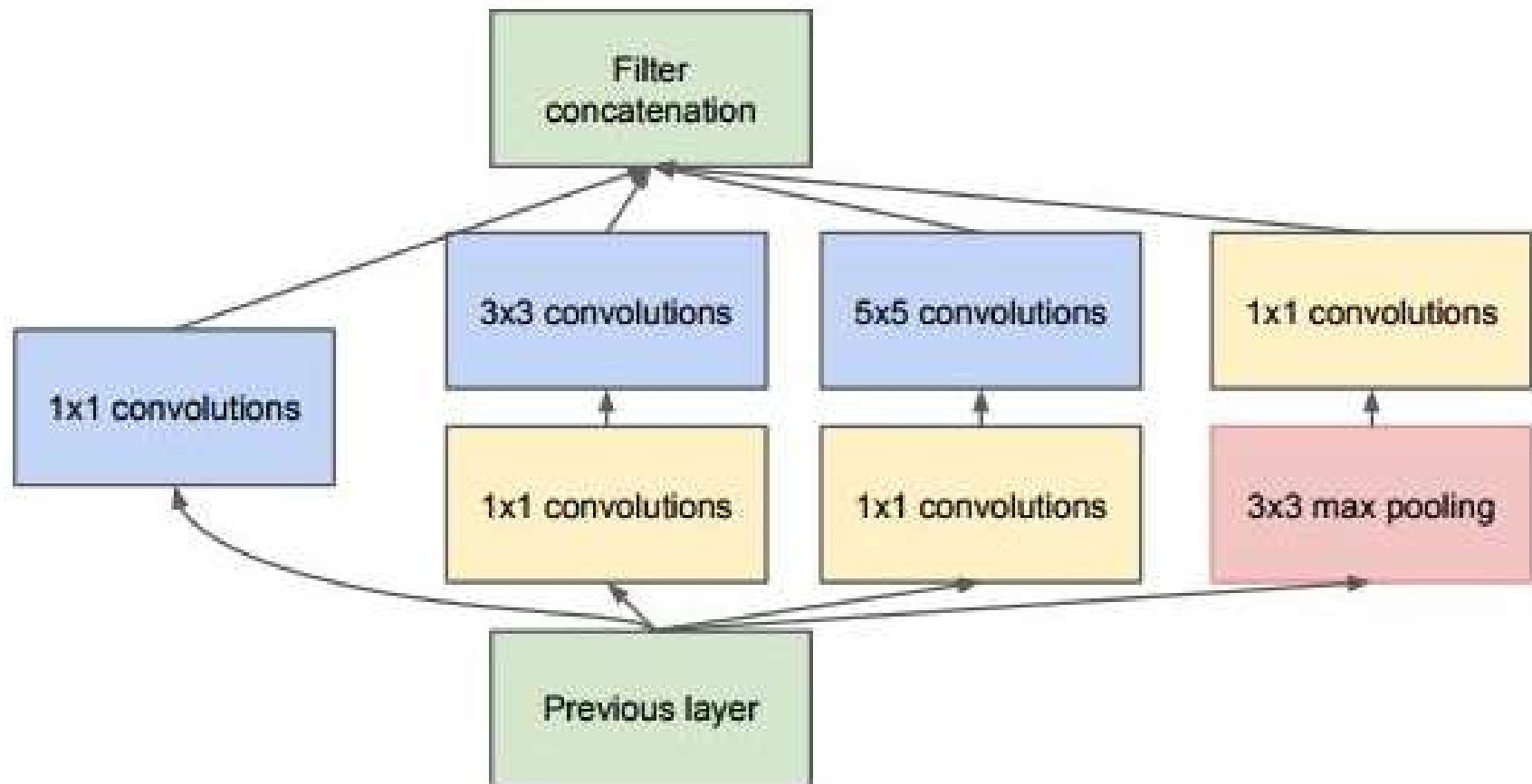


INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

GoogleNet

Módulo Inception con Reducción de Dimensionalidad



Comparativa de Arquitecturas Históricas

Por parámetros y accuracy/error

- LeNet-5 (1998): 0.06M params · MNIST: ~99%
- AlexNet (2012): 60M params · top-5: 15.3%
- VGG-16 (2014): 138M params · top-5: 7.4%
- VGG-19 (2014): 144M params · top-5: 7.3%
- GoogLeNet (2014): 5M params · top-5: 6.67%

Tendencias observadas

- Más profundidad → mejor accuracy (hasta cierto límite)
- GoogLeNet: misma o mejor accuracy que VGG con 28X menos params
- Las capas FC son el cuello de botella de parámetros

Comparativa de Arquitecturas Históricas

Lecciones aprendidas para el diseño de arquitecturas

- Filtros pequeños (3×3) + más capas superan a filtros grandes
- La reducción de canales (1×1 bottleneck) es clave para la eficiencia
- Global Average Pooling (GAP) elimina la dependencia de la resolución y reduce overfitting
- La profundidad ayuda — pero es necesario resolver el vanishing gradient

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Arquitecturas Modernas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

ResNet — Skip Connections (He et al., 2016)

- Ejemplo: Consideremos la siguiente situación:
 - A una red de 20 capas que funciona bien y le agregamos 36 capas
 - ¿Qué deberían hacer esas 36 capas extras para no empeorar el resultado? Respuesta obvia: no hacer nada, simplemente pasar la señal intacta, es decir, cada capa extra debería aprender la función identidad $H(x) = x$
 - Degradation problem;: SGD falla en encontrar esa respuesta aunque matemáticamente existe
 - Solución de ResNet: no pedirle al bloque que aprenda la identidad — en cambio, la identidad se conecta directamente por el skip connection y el bloque solo aprende la corrección sobre ella

ResNet — Skip Connections

La idea central

- La salida del bloque residual es la suma del residuo aprendido $F(x, \{W_i\})$ y la entrada x directamente

$$y = F(x, \{W_i\}) + x$$

- El bloque aprende el residuo $F(x)$ en lugar de la transformación completa $H(x)$

Flujo del gradiente

- El gradiente de la pérdida L respecto a la activación x_1 de la capa l

$$\frac{\partial L}{\partial x_l} = \frac{\partial L}{\partial x_L} \cdot \left(1 + \frac{\partial}{\partial x_l} \sum_i F(x_i, \{W_i\}) \right)$$

contiene el término "+1" que garantiza que el gradiente llega intacto a las capas tempranas, incluso si $\partial F / \partial x_1 \rightarrow 0$

ResNet — Skip Connections

Proyección cuando las dimensiones cambian

- Si $F(x)$ y x tienen distinto tamaño: shortcut = Conv(1×1 , S=stride)
- Esto alinea las dimensiones para la suma

Impacto histórico

- Permite entrenar redes de 50, 101, 152 capas de forma estable
- ILSVRC 2015: top-5 error 3.57% — primer sistema que supera a humanos (5.1%)
- Skip connections se convirtieron en estándar: BERT, GPT, ViT las usan todas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

ResNet — BasicBlock y Bottleneck

BasicBlock (ResNet-18, ResNet-34)

- $\text{Conv}(3 \times 3, \text{BN}, \text{ReLU}) \rightarrow \text{Conv}(3 \times 3, \text{BN}) \rightarrow (+x) \rightarrow \text{ReLU}$
- 2 convoluciones de 3×3 con BN entre ellas
- Parámetros por bloque (C canales): $2 \times C^2 \times 9 = 18C^2$

Bottleneck (ResNet-50, 101, 152)

- $\text{Conv}(1 \times 1, \text{BN}, \text{ReLU}) \rightarrow \text{Conv}(3 \times 3, \text{BN}, \text{ReLU}) \rightarrow \text{Conv}(1 \times 1, \text{BN}) \rightarrow (+x) \rightarrow \text{ReLU}$
- El primer 1×1 reduce canales: $C \rightarrow C/4$
- La $\text{Conv}(3 \times 3)$ opera en el espacio reducido de canales
- El último 1×1 restaura: $C/4 \rightarrow C$

Parámetros por bloque Bottleneck

- $C \times (C/4 \times 1) + (C/4) \times (C/4) \times 9 + (C/4) \times C \times 1 = C^2/4 + 9C^2/16 + C^2/4 \approx 0.7C^2$
- Reducción $\sim 26X$ comparado con $2 \times \text{Conv}(3 \times 3)$ en espacio completo

Intuición

- Comprimir $\rightarrow$ procesar en espacio comprimido $\rightarrow$ descomprimir
- El cuello de botella fuerza representaciones eficientes

ResNet — Familia de Modelos

Modelos estándar

- ResNet-18: 18 capas · 11.7M params · BasicBlock · top-1: 69.8%
- ResNet-34: 34 capas · 21.8M params · BasicBlock · top-1: 73.3%
- ResNet-50: 50 capas · 25.6M params · Bottleneck · top-1: 76.1%
- ResNet-101: 101 capas · 44.5M params · Bottleneck · top-1: 77.4%
- ResNet-152: 152 capas · 60.2M params · Bottleneck · top-1: 78.3%

ResNet-50 como baseline estándar

- 25.6M params: manejable en GPU de 8GB para batch=32
- Buen equilibrio accuracy/velocidad/parámetros
- Pesos pre-entrenados excelentes para transfer learning

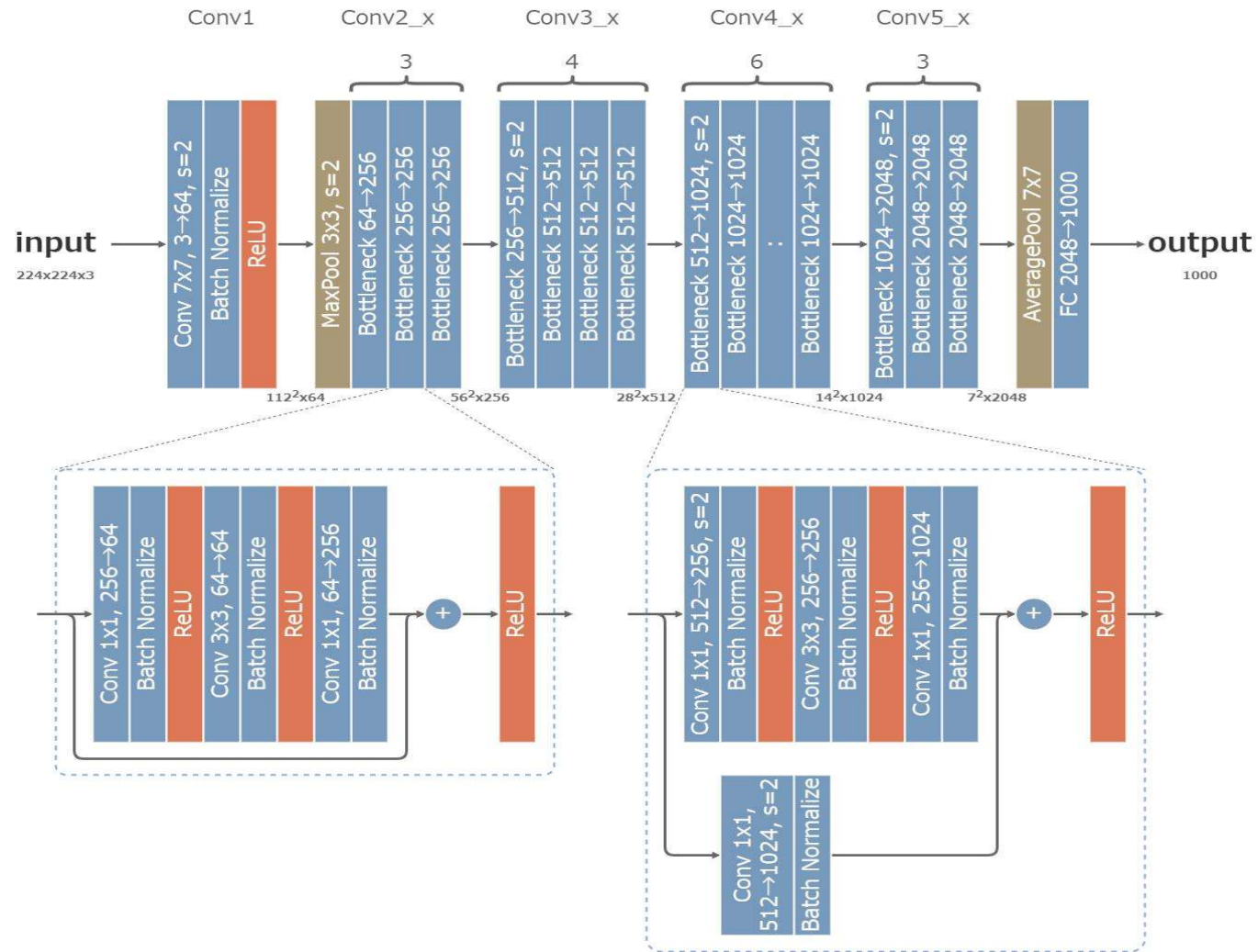
Variantes mejoradas

- ResNetV2: BN → ReLU → Conv (pre-activation) — mejor flujo de gradiente
- ResNeXt: agrega cardinalidad (grupos de canales) — mismo costo, mejor accuracy
- Wide ResNet: más ancho, menos profundo — entrenamiento más eficiente

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

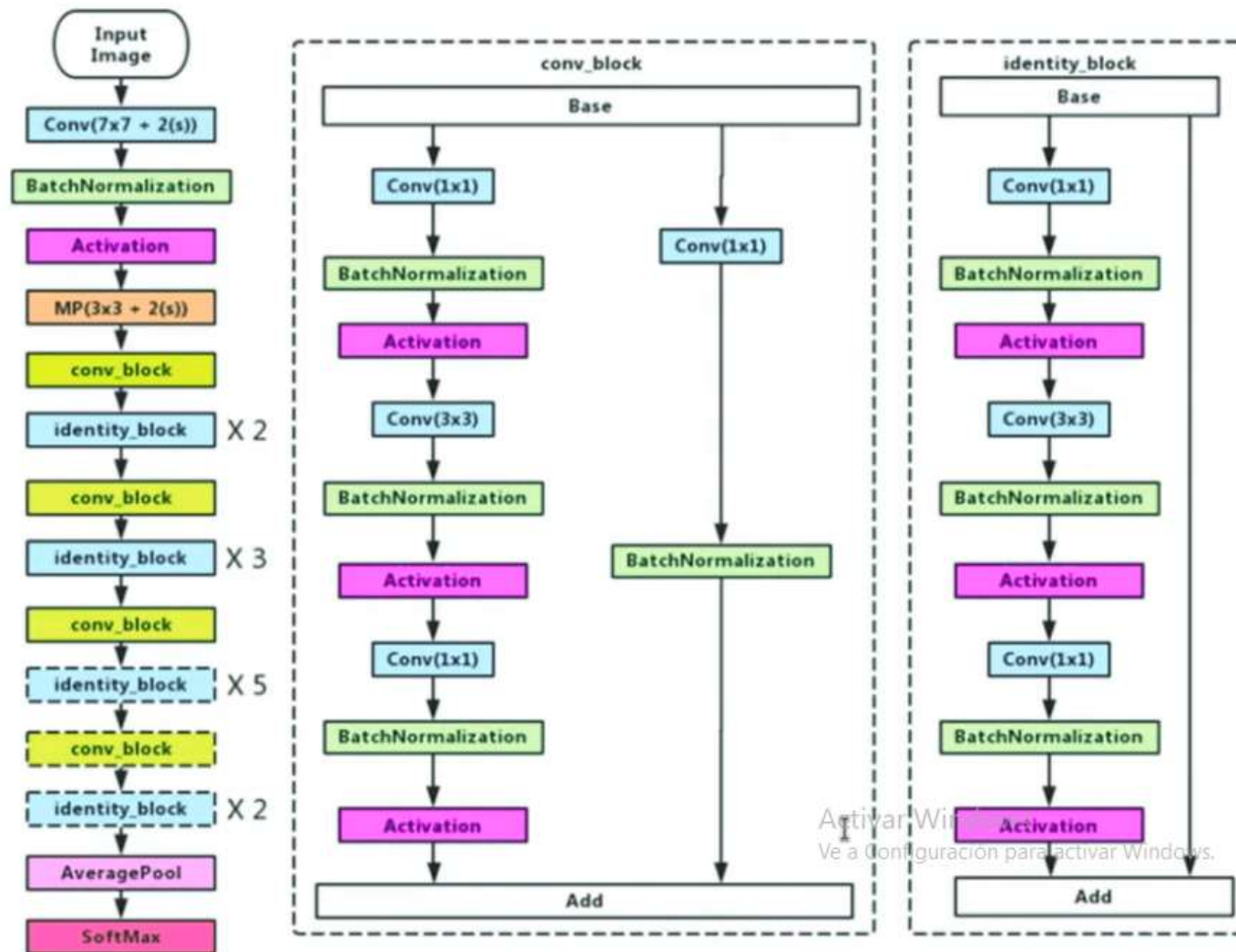
ResNet-50



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

ResNet-50



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

DenseNet (Huang et al., 2017)

Idea: conexiones densas

- Cada capa se conecta con TODAS las capas anteriores (no solo la inmediata)
- Capa l recibe: $\text{concat}(x_0, x_1, \dots, x_{l-1})$

Ventajas sobre ResNet

- Flujo de gradiente máximo: cada capa accede directamente a la pérdida
- Reutilización de features: no necesita reaprender representaciones previas
- Regularización: los feature maps de capas tempranas se reusan directamente

Dense Block

- k capas, cada una con $\text{BN} \rightarrow \text{ReLU} \rightarrow \text{Conv}(3 \times 3)$
- k es el 'growth rate': cada capa añade k canales al concatenado
- Típico $k = 32$: capa 10 recibe $9 \times 32 + C_{\text{input}}$ canales de entrada

Transición entre bloques

- $\text{Conv}(1 \times 1)$: reduce canales (compresión) · $\text{AvgPool}(2 \times 2)$: baja resolución

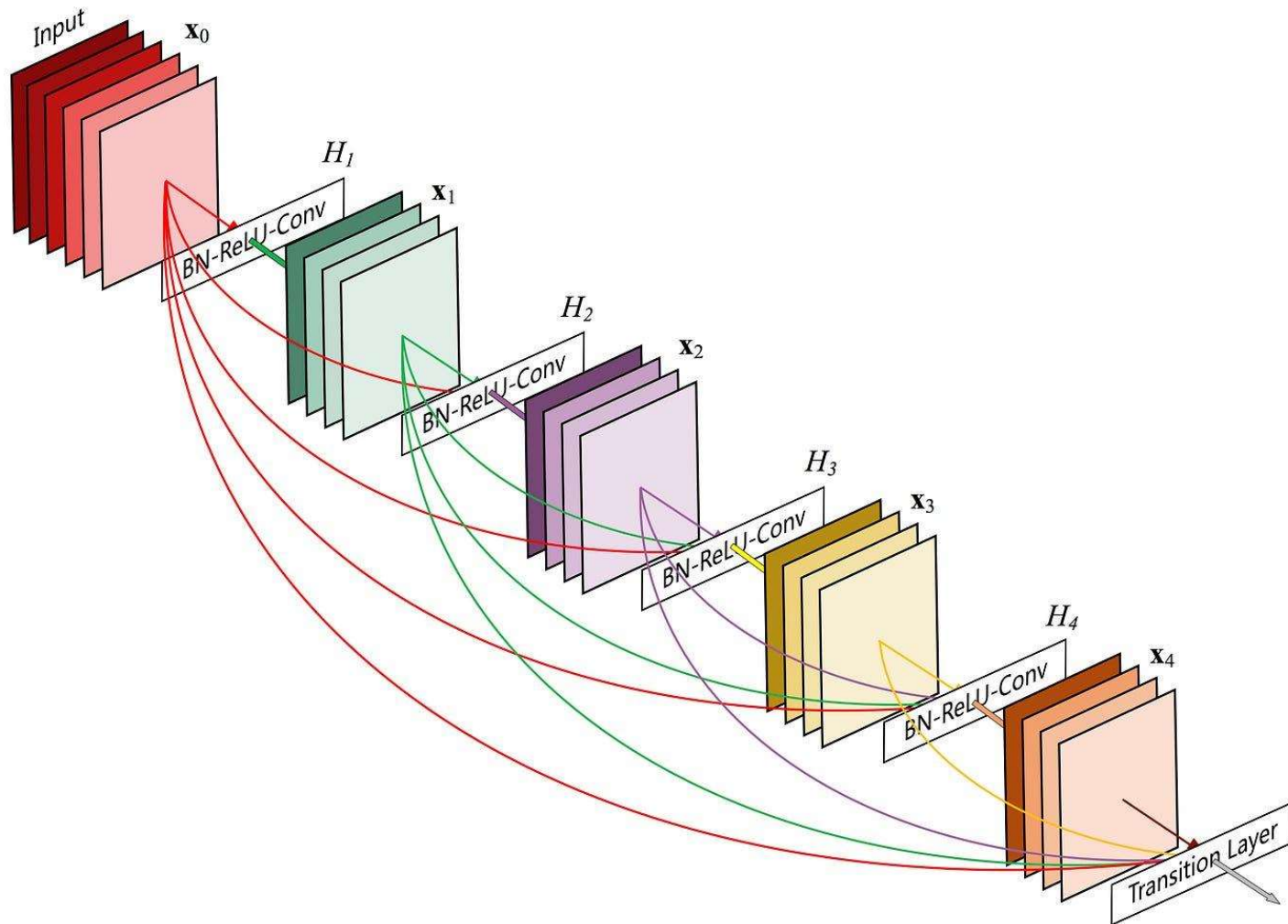
En producción

- DenseNet-121: 8M params, 74.7% top-1 — muy eficiente en params
- Popular en imágenes médicas por reutilización agresiva de features

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

DenseNet



SENet — Squeeze-and-Excitation (Hu et al., 2018)

ILSVRC 2017 winner

- Añade atención por canal a cualquier arquitectura CNN existente

Mecanismo SE block

1. Squeeze — Global Average Pooling por canal

Para cada canal c , promediar la feature map $H \times W$:

$$z_c = \frac{1}{HW} \sum_{i,j} x_c(i,j)$$

2. Excitation — reescalado por canal

Con ratio de reducción r , la importancia de cada canal es ($\delta = ReLU$, $\sigma = Sigmoid$):

$$s = \sigma(W_2 \cdot \delta(W_1 \cdot z))$$

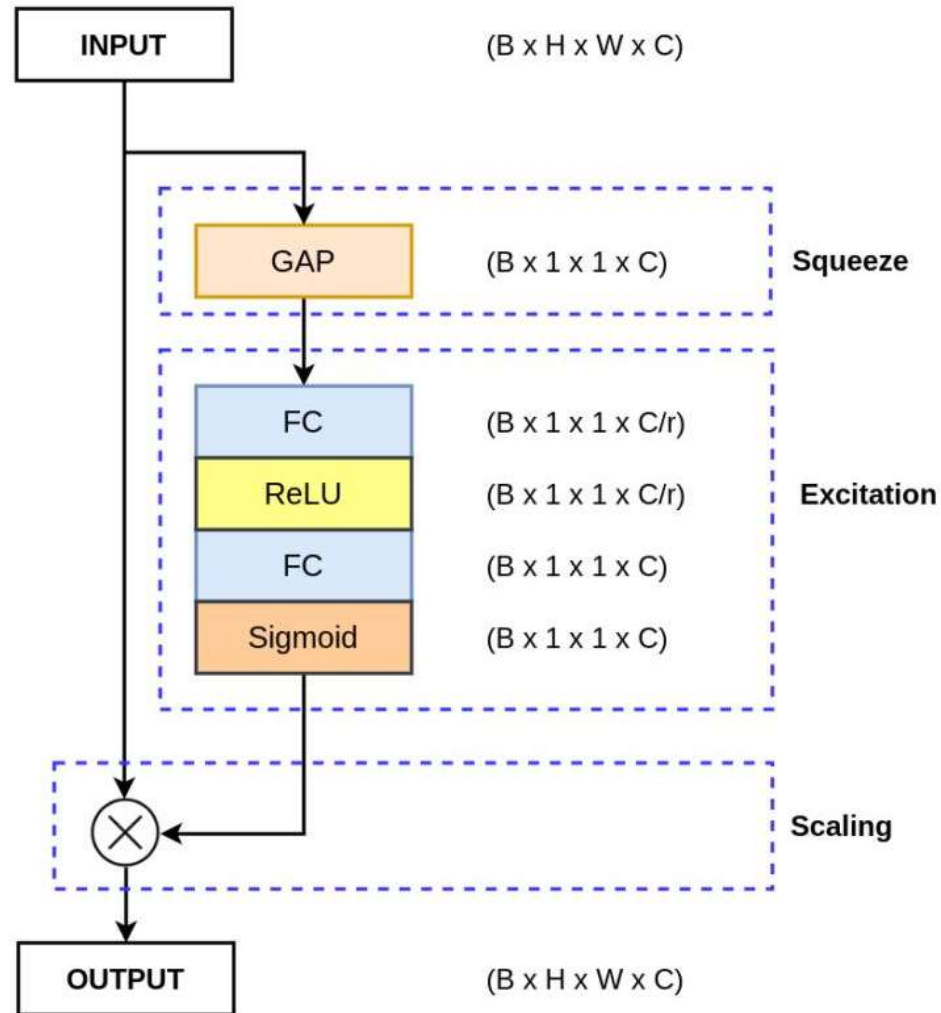
3. Recalibración del feature map canal a canal:

$$\tilde{x}_c = s_c \cdot x_c$$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

SENet — Squeeze-and-Excitation



SENet — Squeeze-and-Excitation

El ratio de reducción r

- Es un hiperparámetro que controla cuánto se comprimen los canales dentro del bloque SE antes de calcular la importancia de cada canal
- La primera red FC reduce de C canales a C/r , y el segundo los restaura a C . El ratio r determina el tamaño de ese cuello de botella intermedio
- Típicamente $r = 16$: reduce de C a $C/16$ en la primera FC
- Con $C = 256$: $256 \times 16 + 16 \times 256 = 8,192$ parámetros extra; costo muy bajo

¿Por qué funciona?

- Aprende qué canales son informativos para cada imagen
- Atención selectiva: amplifica señales relevantes, atenúa ruido

Impacto

- SE-ResNet-50: mejora 1.3% top-1 con solo 10% más parámetros
- Se integra en EfficientNet (MBConv+SE), MobileNetV3

MobileNet (Howard et al., 2017)

Objetivo: CNNs eficientes para dispositivos móviles

- Inferencia en CPU de smartphone — latencia <100ms

Ingrediente clave: Depthwise Separable Convolution

- Reemplaza $\text{Conv}(C_{in} \rightarrow C_{out}, F \times F)$ por:
 - Depthwise: C_{in} filtros $F \times F$ independientes por canal
 - Pointwise: $\text{Conv}(1 \times 1, C_{in} \rightarrow C_{out})$ para mezclar canales

Reducción de cómputo

- FLOP estándar: $H \times W \times C_{in} \times C_{out} \times F^2$
- FLOP depthwise sep: $H \times W \times C_{in} \times F^2 + H \times W \times C_{in} \times C_{out}$
- Ratio: $1/C_{out} + 1/F^2 \approx 1/8 - 1/9$ para $C_{out} = 256, F = 3$

Hiperparámetros de escalado

- Width multiplier α : escala canales uniformemente ($\alpha = 0.25, 0.5, 0.75, 1.0$)
- Resolution multiplier ρ : escala la resolución de entrada (224, 192, 160, 128)

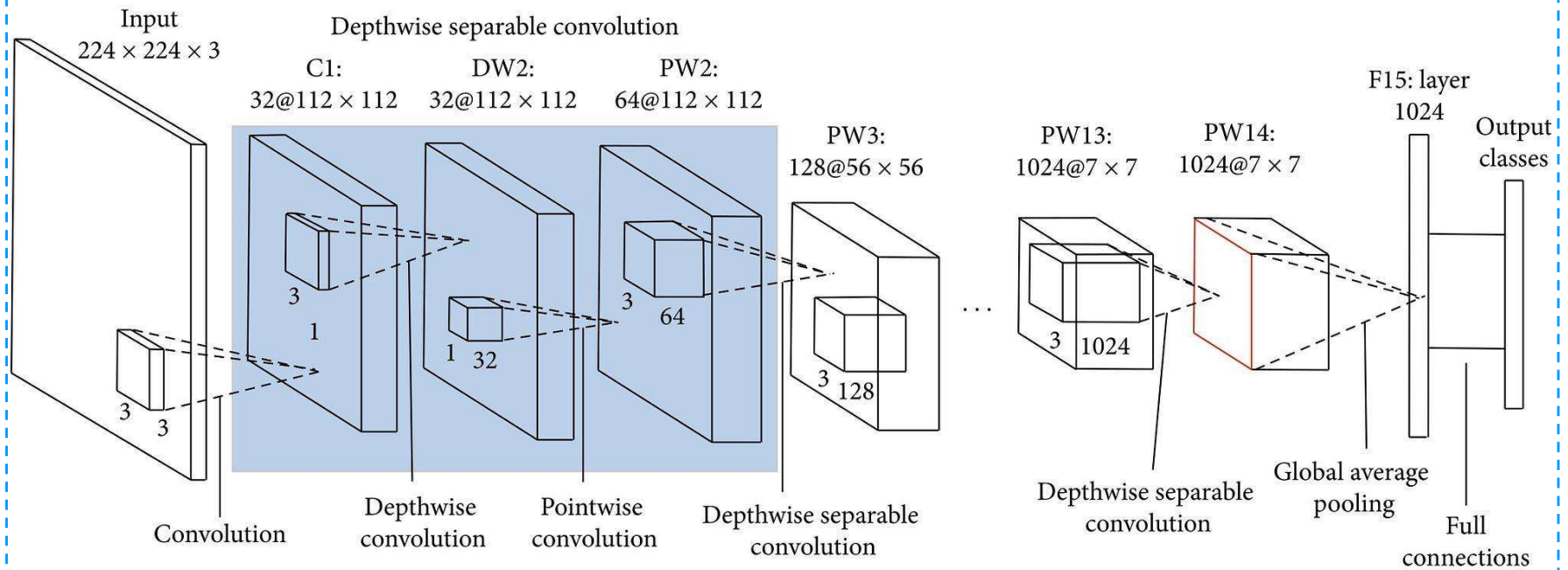
Resultados MobileNetV1

- 72.9% top-1 con 4.2M params y 569M FLOP vs AlexNet (15.3% top-5, 60M params)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

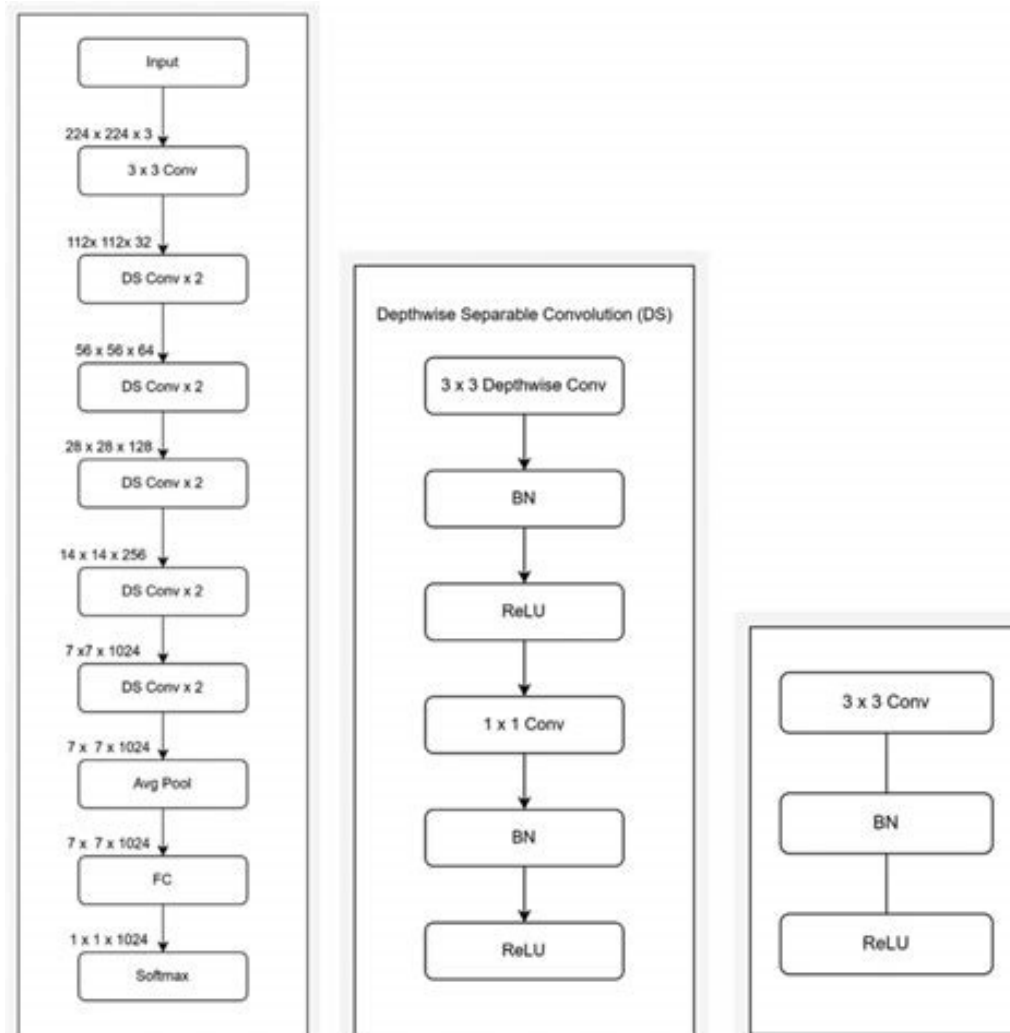
MobileNet (Howard et al., 2017)



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

MobileNet (Howard et al., 2017)



INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

MobileNetV2 — Inverted Residuals

Problema de MobileNetV1

- Los canales se comprimen antes de la conv densa → información se pierde

Inverted Residual Block

- Al revés que el bottleneck de ResNet: expand → process → compress
- Conv(1×1 , $C \rightarrow tC$, ReLU6) ← expand (factor $t=6$)
- DepthwiseConv($tC \rightarrow tC$, 3×3 , ReLU6) ← procesar en espacio expandido
- Conv(1×1 , $tC \rightarrow C$) ← compress sin activación

Skip connection solo cuando $C_{in} = C_{out}$

- Si stride=1: residual connection directa
- Si stride=2 (downsampling): sin residual

Por qué sin ReLU en la última 1×1

- ReLU destruye información en espacios de baja dimensión (canales comprimidos)
- Teorema de capacidad de representación lineal: ReLU en espacio pequeño = pérdida

MobileNetV2 en producción

- 72.0% top-1 con 3.4M params y 300M FLOP
- Usado como backbone en SSDLite, DeepLabV3+, etc.

EfficientNet — Motivación y Compound Scaling

Tan & Le (Google Brain, 2019)

- Pregunta: ¿cómo escalar una CNN óptimamente con más recursos?
- Opciones anteriores: solo escalar profundidad, solo anchura, o solo resolución

Compound Scaling — las tres dimensiones juntas

- $d = \alpha^\varphi$ (profundidad: más capas)
- $w = \beta^\varphi$ (anchura: más canales por capa)
- $r = \gamma^\varphi$ (resolución: imágenes más grandes)
- Restricción: $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ (control de FLOP totales)

Coeficientes óptimos (búsqueda NAS con $\varphi=1$)

- $\alpha = 1.2, \beta = 1.1, \gamma = 1.15 \rightarrow$ EfficientNet-Bn usa $\varphi = n$

Por qué escalar juntas las tres dimensiones

- Mayor resolución requiere más capas (más RF) y más canales
- Las tres dimensiones están relacionadas — escalar una sola es subóptimo

Resultado

- EfficientNet-B7: 84.3% top-1 con 66M params
- Mejor accuracy/FLOP que cualquier CNN anterior (2019)

EfficientNet — Arquitectura MBConv

Bloque base: MBConv (Mobile Inverted Bottleneck + SE)

- $\text{Conv}(1 \times 1, \text{expand}) \rightarrow \text{DepthwiseConv}(F \times F) \rightarrow \text{SE block} \rightarrow \text{Conv}(1 \times 1, \text{project})$
- Con skip connection si $\text{stride}=1$ y $C_{\text{in}}=C_{\text{out}}$

SE block integrado

- $r=4$ (reducción de canales): mayor atención que en SE original ($r=16$)

Arquitectura EfficientNet-B0 (baseline NAS)

- Stem: $\text{Conv}(3 \times 3, S=2, 32 \text{ canales})$
- Bloque 1: MBConv1, $3 \times 3, \times 1, 16\text{ch}$
- Bloque 2: MBConv6, $3 \times 3, \times 2, 24\text{ch}$
- Bloque 3: MBConv6, $5 \times 5, \times 2, 40\text{ch}$
- Bloque 4: MBConv6, $3 \times 3, \times 3, 80\text{ch}$
- Bloque 5: MBConv6, $5 \times 5, \times 3, 112\text{ch}$
- Bloque 6: MBConv6, $5 \times 5, \times 4, 192\text{ch}$
- Bloque 7: MBConv6, $3 \times 3, \times 1, 320\text{ch}$
- Head: $\text{Conv}(1 \times 1, 1280) \rightarrow \text{GAP} \rightarrow \text{Dropout} \rightarrow \text{FC}(1000)$

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

EfficientNet — Familia B0–B7 y V2

Familia EfficientNet-B0 a B7

- B0: 224px, 5.3M, 77.1% · B4: 380px, 19M, 83.0%
- B1: 240px, 7.8M, 79.1% · B5: 456px, 30M, 83.6%
- B2: 260px, 9.1M, 80.1% · B6: 528px, 43M, 84.0%
- B3: 300px, 12M, 81.6% · B7: 600px, 66M, 84.3%

EfficientNetV2 (Tan & Le, 2021)

- Mismo compound scaling pero con bloques Fused-MBConv en capas tempranas
- Fused-MBConv: Conv(3×3) + Conv(1×1) — sin depthwise — más rápido en GPU
- Entrenamiento 11× más rápido que EfficientNet-B7 para igual accuracy

EfficientNetV2-S/M/L

- V2-S: 21.5M params, 83.9% top-1 · V2-M: 54M, 85.1% · V2-L: 119M, 85.7%

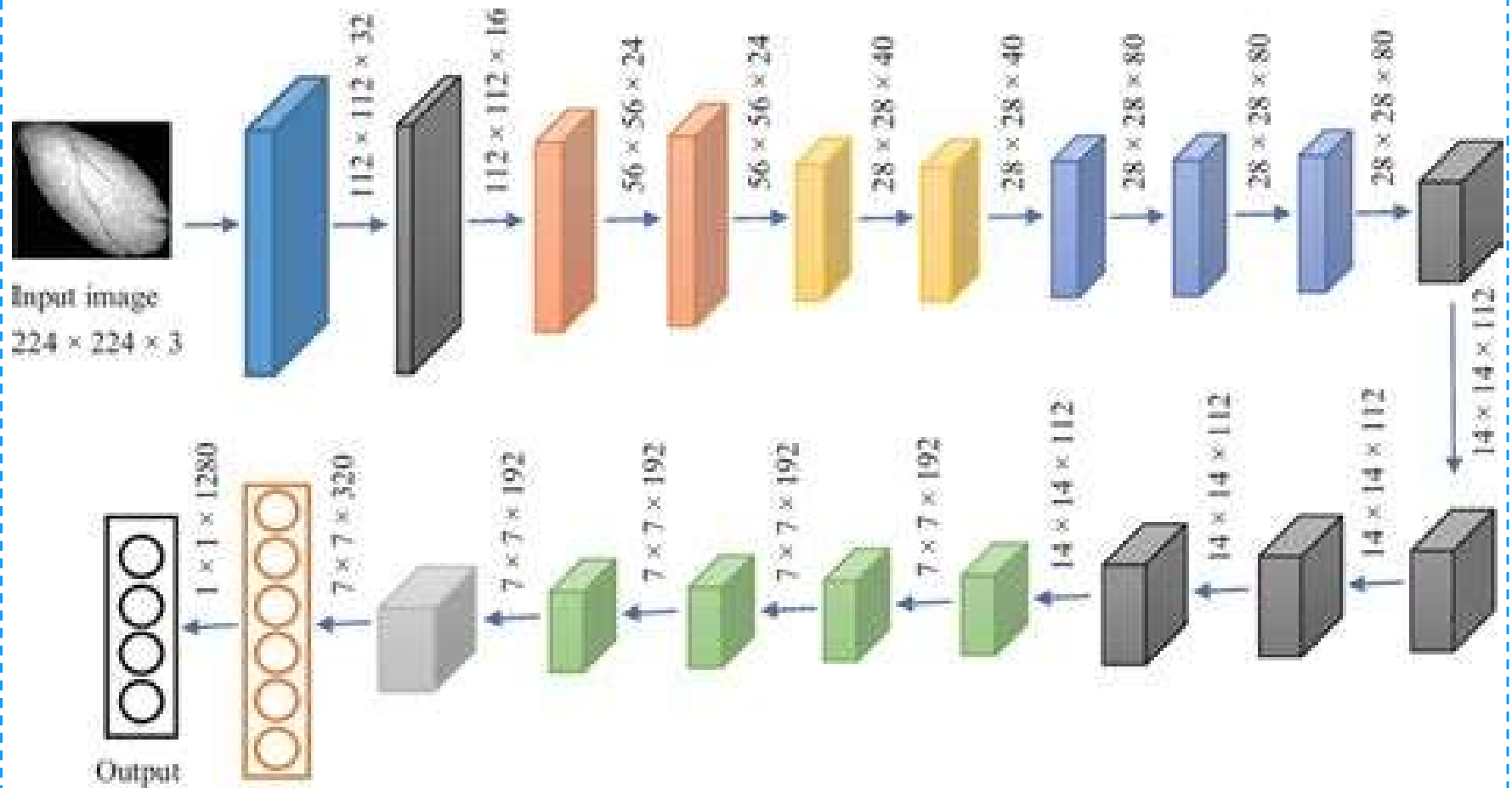
Recomendación práctica

- EfficientNetV2-S: excelente punto de partida para producción
- Mejor accuracy/FLOP que ResNet-50 con entrenamiento más rápido

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

EfficientNet



ConvNeXt — Contexto y Motivación

Liu et al. (Meta AI + CMU, 2022)

- Pregunta: ¿pueden las CNNs puras igualar a los Vision Transformers?
- Vision Transformers (ViT, Swin) dominaban los benchmarks en 2021

Estrategia: modernizar ResNet paso a paso

- Partir de ResNet-50 y aplicar cada mejora de los ViTs de forma aislada
- Medir el impacto de cada cambio → entender qué importa y qué no

Los 5 grupos de cambios

1. Macro design (stage ratios, stem design)
2. Bloques ResNeXt (depthwise convolutions)
3. Diseño de filtros invertidos y tamaño de filtro
4. Micro design (activaciones, normalización)
5. Separación de stages en downsampling

Resultado final

- ConvNeXt-T: 28M params, 82.1% top-1 — igual que Swin-T
- ConvNeXt-B: 89M params, 85.1% — comparable a ViT-B
- Demuestra que las CNNs no estaban obsoletas, sino infradiseñadas

ConvNeXt — Cambios de Diseño Detallados

Cambio 1: Patchify stem

- ResNet: $\text{Conv}(7 \times 7, S=2) + \text{MaxPool}(3 \times 3, S=2) \rightarrow 56 \times 56$
- ConvNeXt: $\text{Conv}(4 \times 4, S=4) \rightarrow 56 \times 56$ — mismo resultado, más simple

Cambio 2: Stage ratios y distribución de bloques

- ResNet-50: stages (3,4,6,3) $\rightarrow$ ConvNeXt: (3,3,9,3) — como Swin

Cambio 3: Depthwise Conv(7×7)

- Reemplaza $\text{Conv}(3 \times 3)$ por $\text{DepthwiseConv}(7 \times 7)$: mismo cómputo, RF mayor
- 7×7 depthwise: $RF = 7$ con el costo de parámetros de 3×3 depthwise

Cambio 4: Bottleneck invertido (ratio 4×)

- Channels: $C \rightarrow 4C \rightarrow C$ — como Transformer FFN (4X expansión)

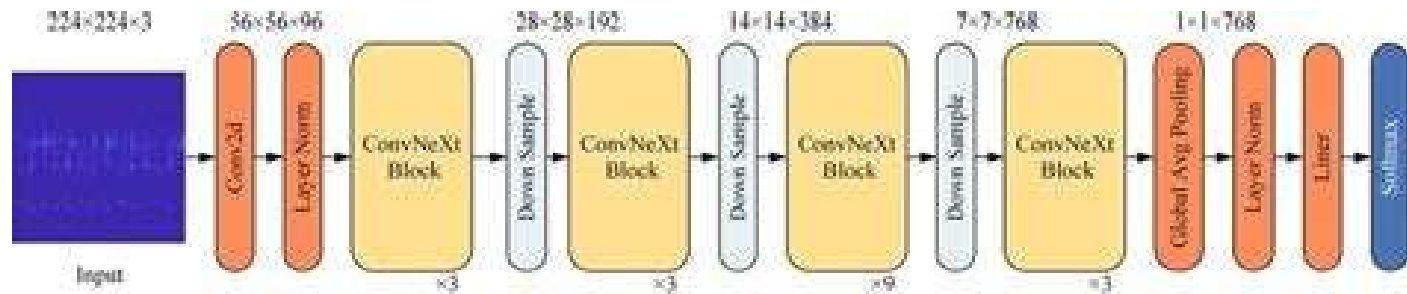
Cambio 5: Normalización y activación

- BN $\rightarrow$ LayerNorm: más estable, sin dependencia del batch
- ReLU $\rightarrow$ GELU: suave, diferenciable, mejor en espacios expandidos
- Solo 1 activación y 1 normalización por bloque (menos que ResNet)

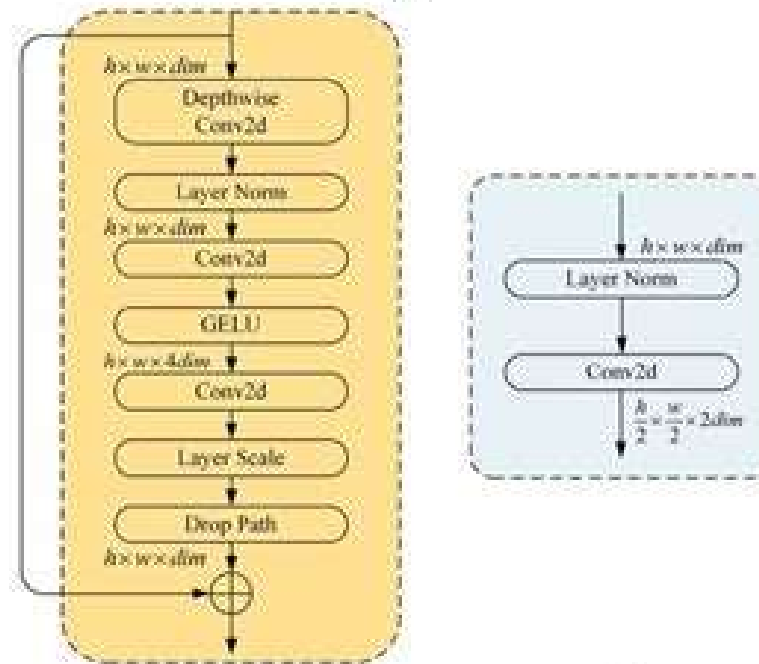
INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

ConvNeXt



(a)



ConvNeXt — Resultados y Comparativa

Comparativa en ImageNet-1K (top-1 accuracy)

- ResNet-50: 25.6M · 76.1% · baseline
- Swin-T: 28.3M · 81.3% · ViT con ventanas locales
- ConvNeXt-T: 28.6M · 82.1% · +0.8% sobre Swin-T
- Swin-B: 88.0M · 83.5%
- ConvNeXt-B: 88.6M · 85.1% · +1.6% sobre Swin-B
- ConvNeXt-XL: 350M · 87.0% (ImageNet-22K pre-training)

Conclusiones

- Las mejoras de los ViTs son diseño, no atención — son transferibles a CNNs
- LN > BN para escalado a datos grandes
- Depthwise conv grande (7×7) > conv estándar pequeña (3×3)

ConvNeXt V2 (2023)

- Añade Global Response Normalization (GRN) y pre-entrenamiento MAE
- ConvNeXt-V2-H: 88.9% top-1 (ImageNet-22K) — estado del arte CNNs puras

Cómo Elegir Arquitectura

Criterios principales

- Dataset pequeño (<1K): ResNet-18/34 — pocas params, fácil de fine-tuning
- Dataset mediano: ResNet-50 — baseline universal, ampliamente estudiado
- Máxima accuracy: ConvNeXt-B/L o EfficientNetV2-L
- Recursos limitados (móvil/edge): MobileNetV3, EfficientNet-B0

Por velocidad de inferencia

- GPU: ResNet-50 ~ 4ms/imagen · EfficientNet-B4 ~ 8ms · ConvNeXt-B ~ 6ms
- CPU: MobileNetV3 ~ 10ms · ResNet-18 ~ 80ms · ResNet-50 ~ 250ms

Por dominio

- Imágenes médicas: DenseNet (reutilización de features, datos limitados)
- Detección de objetos: ResNet + FPN, EfficientDet
- Segmentación: DeepLab + ResNet, UNet + cualquier encoder

Recomendación universal 2024

- Empezar con ResNet-50 pre-entrenado en ImageNet: baseline sólido y comprensible
- Escalar a ConvNeXt o EfficientNetV2 si se necesita más accuracy

Transferencia de Aprendizaje (Transfer Learning)

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Qué Aprenden las Capas de una CNN

Visualización de Zeiler & Fergus (2014) — DeconvNet

- Técnica para proyectar activaciones de vuelta al espacio de píxeles

Capas bajas (1-2): features genéricas

- Detectores de bordes orientados (Gabor-like: diagonal, horizontal, vertical)
- Detectores de color: transiciones rojo-verde, azul-amarillo
- Blobs de frecuencia espacial: alta/baja frecuencia

Capas medias (3-4): features semi-específicas

- Texturas: mallas, patrones repetitivos, piel, pelaje
- Esquinas y uniones de bordes
- Partes simples de objetos: ruedas, ventanas, ojos

Capas altas (5+): features específicas del dominio

- Detectores de caras, perros, coches — muy específicos de ImageNet
- Representaciones semánticas de alto nivel

Implicación para transfer learning

- Capas bajas: reusar siempre (universales para cualquier imagen natural)
- Capas altas: reemplazar o fine-tune para el nuevo dominio

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

¿Por qué Funciona el Transfer Learning?

La hipótesis de representaciones universales

- Features de capas bajas de redes entrenadas en ImageNet son útiles para cualquier tarea de visión con imágenes naturales — evidencia empírica masiva

Pre-entrenamiento en ImageNet como inicialización

- ImageNet-1K: 1.2M imágenes, 1000 clases — diversidad visual enorme
- La red aprende una distribución de features rica y general
- Fine-tuning desde esta inicialización >> entrenamiento desde cero

Cuando los datos son limitados

- Con 100 imágenes por clase: transfer learning puede superar en 20-30% accuracy a entrenar desde cero con el mismo dataset

Casos donde el transfer puede no ayudar

- Dominio muy diferente (imágenes de satélite, imágenes médicas de alta resolución)
- En estos casos: pre-entrenar en dominio propio o usar pesos parcialmente

Pre-entrenamiento alternativo

- CLIP (contrastivo imagen-texto): mejora transfer a tareas conceptuales
- MAE (Masked Autoencoders): auto-supervisado — eficaz con pocos datos etiquetados

Feature Extraction

Definición

- Congelar todos los pesos de la red pre-entrenada
- Usar la red como extractor de features fijo
- Solo entrenar la nueva cabeza de clasificación (FC final)

Cuándo usar feature extraction

- Dataset muy pequeño (<500 imágenes por clase)
- Dominio similar al pre-entrenamiento (fotos naturales)
- Recursos computacionales limitados

Variante: extraer features offline

- Pasar todo el dataset una vez por la red congelada → guardar embeddings
- Entrenar un clasificador simple (SVM, LogReg) sobre los embeddings
- Extremadamente rápido: el paso costoso es solo la extracción inicial

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Fine-tuning

Definición

- Descongelar parte o toda la red y reentrenar end-to-end
- LR pequeño para no destruir los pesos pre-entrenados

Cuándo usar fine-tuning

- Dataset mediano-grande (>1K imágenes por clase)
- Dominio diferente al pre-entrenamiento
- Se quiere maximizar la accuracy final

Riesgo: catastrophic forgetting

- Un LR demasiado alto destruye las representaciones pre-entrenadas
- Los pesos buenos de ImageNet se sobrescriben con ruido del dataset pequeño

Estrategia segura

1. Entrenar solo la cabeza (5-10 épocas) hasta convergencia
2. Descongelar capas superiores con LR 10 – 100X menor que la cabeza
3. Opcionalmente: descongelar más capas progresivamente

Elección del LR para fine-tuning

- Regla práctica: $LR_{finetune} = LR_{head} / 10$ a $LR_{head} / 100$
- Con AdamW: $LR = 1e - 5$ a $1e - 4$ es típico para capas pre-entrenadas

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Estrategias Según Dataset y Dominio

Matriz de decisión

- Dataset pequeño + dominio similar → Feature extraction (solo entrenar FC)
- Dataset pequeño + dominio diferente → Fine-tune capas altas + FC
- Dataset grande + dominio similar → Fine-tune toda la red (LR bajo)
- Dataset grande + dominio diferente → Fine-tune toda la red o entrenar desde cero

Dataset pequeño: por qué no fine-tune completo

- Con pocos datos, el fine-tune sobreajusta rápidamente
- Las capas bajas (bordes, texturas) son transferibles — no necesitan ajuste

Dataset diferente: descongelar capas altas

- Las capas altas son específicas de ImageNet (gatos, perros, coches)
- Para imágenes médicas: las últimas 2-3 capas deben readaptarse al nuevo dominio

Indicador de cuántas capas descongelar

- Monitorear val_loss durante fine-tune progresivo
- Si sigue bajando al descongelar más. Si sube: parar

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Pesos Pre-entrenados en PyTorch

torchvision.models (desde v0.13)

- `from torchvision.models import ResNet50_Weights`
- `model = models.resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)`

Pesos disponibles por modelo

- IMAGENET1K_V1: entrenamiento estándar (accuracy histórica)
- IMAGENET1K_V2: recetas mejoradas (+3-4% top-1) — usar siempre

Normalization automática

- `preprocess = ResNet50_Weights.IMAGENET1K_V2.transforms()`
- Incluye resize, crop, normalize con media/std de ImageNet

Modelos disponibles en 2024

- `resnet18/34/50/101/152`, `efficientnet_b0..b7`, `convnext_tiny/small/base`
- `vit_b_16`, `vit_l_16`, `swin_b` — también Transformers

Alternativa: timm (PyTorch Image Models)

- `import timm; model = timm.create_model('convnext_base', pretrained=True)`
- Más de 900 modelos pre-entrenados con una sola línea

LRs Diferenciales por Grupo de Capas

Motivación

- Diferentes partes de la red deben actualizarse a velocidades diferentes
- Capas pre-entrenadas: LR pequeño para preservar features ImageNet
- Cabeza nueva: LR grande para aprender rápido

Implementación en PyTorch

```
optimizer = torch.optim.AdamW([
    {'params': model.layer1.parameters(), 'lr': 1e-5},
    {'params': model.layer2.parameters(), 'lr': 1e-5},
    {'params': model.layer3.parameters(), 'lr': 5e-5},
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(), 'lr': 1e-3},
], weight_decay=1e-2)
```

Discriminative fine-tuning (ULMFiT, Howard & Ruder, 2018)

- Ratio típico: $LR_{\{capa\ l\}} = LR_{\{l+1\}} / 2.6$
- Capas más tempranas: LR menor · Cabeza: LR máximo

Fine-tuning Progresivo y Cosine Annealing

Gradual unfreezing (Howard & Ruder, 2018)

- Fase 1: solo FC → Fase 2: + últimas capas → Fase 3: + toda la red
- Cada fase converge antes de descongelar más capas

Combinación con Cosine Annealing with Warm Restarts

- LR sube al inicio de cada fase (warm restart) y baja siguiendo un coseno

$$\eta_t = \eta_{min} + 0.5(\eta_{max} - \eta_{min}) \left(1 + \cos \left(\frac{\pi t}{T} \right) \right)$$

Ciclos de fine-tuning recomendados

- Ciclo 1 (solo FC): $\eta_{max} = 1e-3$, $T = 5$ épocas
- Ciclo 2 (+ layer4): $\eta_{max} = 1e-4$, $T = 5$ épocas
- Ciclo 3 (red completa): $\eta_{max} = 1e-5$, $T = 10$ épocas

One Cycle LR (Smith, 2018) — alternativa muy popular

- `scheduler = OneCycleLR(optimizer, max_lr=1e-3, steps_per_epoch=len(loader), epochs=20)`
- LR sube linealmente hasta `max_lr` y baja siguiendo coseno — muy efectivo

INTELIGENCIA ARTIFICIAL Y APRENDIZAJE PROFUNDO I

Unidad III

Domain Adaptation

Domain gap

- Diferencia estadística entre el dominio fuente (ImageNet) y el objetivo
- Ejemplos: fotos vs rayos X, fotos vs dibujos, fotos de día vs noche

Tipos de domain shift

- Covariate shift: $p(x)$ cambia pero $p(y|x)$ no — los más comunes
- Label shift: $p(y)$ cambia — clases balanceadas vs desbalanceadas
- Dataset bias: el dominio fuente tiene sesgos sistemáticos

Técnicas de domain adaptation

- Fine-tuning con datos del dominio objetivo (más directo)
- Adversarial DA: entrenar discriminador dominio fuente vs objetivo
- DANN (Domain Adversarial Neural Network): minimiza distancia entre dominios

Cuando no hay etiquetas en el dominio objetivo

- Unsupervised DA: alinear distribuciones de features entre dominios
- Self-training: generar pseudo-labels con alta confianza y reentrenar
- DINOv2 (Meta, 2023): features auto-supervisados que generalizan entre dominios